



第4章

快速傅里叶变换(FFT)

主要内容

- 直接计算DFT的运算量
- 按时间抽取的FFT
- 按频率抽取的FFT
- N 为组合数的FFT和基四FFT（自学）
- Chirp-z变换（线性调频z变换）（自学）
- FFT应用

引言

- ❑ **FFT: Fast Fourier Transform**
- ❑ 1965年，Cooley-Turky 发表文章《机器计算傅里叶级数的一种算法》，提出FFT算法，解决DFT运算量太大，在实际使用中受限制的问题。
- ❑ **FFT不是一种新的傅里叶变换，只是DFT的一种快速算法，它可以在数量级的意义上提高运算速度。**
- ❑ **FFT的应用：频谱分析、滤波器实现、实时信号处理等。**

- ❑ DSP芯片实现。TI公司的TMS 320c30， 10MHz时钟，基2-FFT1024点FFT时间15ms。



TI芯片TMS320系列

□ 典型应用：信号频谱计算、系统分析等

频谱分析与功率谱计算

$$x(n) \xleftrightarrow{DFT} X(k)$$

系统分析

$$x(n) \rightarrow [h(n)] \rightarrow y(n)$$

$$x(n) \rightarrow [FFT] \rightarrow$$

↓

$$\otimes \rightarrow [IFFT] \rightarrow y(n)$$

↑

$$h(n) \rightarrow [FFT] \rightarrow$$

4.1 直接计算DFT的问题及改进途径

直接计算DFT的计算量

N 点有限长序列 $x(n)$

$$DFT : X(k) = DFT[x(n)] = \sum_{n=0}^{N-1} x(n)W_N^{kn}, 0 \leq k \leq N-1$$

$IDFT :$

$$x(n) = IDFT[X(k)] = \frac{1}{N} \sum_{k=0}^{N-1} X(k)W_N^{-kn}, 0 \leq n \leq N-1$$

$X(k)$ 和 $x(n)$ 的计算量基本相同，只差一个因子 $1/N$ ，故以 $X(k)$ 的计算量为例来讨论运算量。

$$DFT: \quad X(k) = DFT[x(n)] = \sum_{n=0}^{N-1} x(n) W_N^{kn}, 0 \leq k \leq N-1$$

DFT的矩阵方程表示

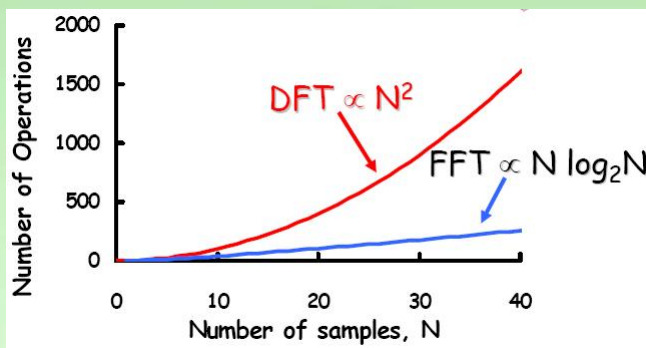
$$\mathbf{X} = \mathbf{W}_N \mathbf{x}$$

$$\begin{bmatrix} X(0) \\ X(1) \\ \vdots \\ X(N-1) \end{bmatrix} = \begin{bmatrix} \overset{\mathbf{n}=0}{W_N^0} & \overset{\mathbf{n}=1}{W_N^0} & \cdots & \overset{\mathbf{n}=N-1}{W_N^0} \\ \overset{\mathbf{k}=1}{W_N^0} & W_N^{1 \times 1} & \cdots & W_N^{(N-1) \times 1} \\ \vdots & \vdots & \ddots & \vdots \\ \overset{\mathbf{k}=N-1}{W_N^0} & W_N^{1 \times (N-1)} & \cdots & W_N^{(N-1) \times (N-1)} \end{bmatrix} \begin{bmatrix} x(0) \\ x(1) \\ \vdots \\ x(N-1) \end{bmatrix}$$

直接计算DFT的计算量

$$\sum_{n=0}^{N-1} x(n) W_N^{nk}$$

	复数乘法	复数加法
一个 $X(k)$	N	$N-1$
N 个 $X(k)$ (N 点 DFT)	N^2	$N(N-1)$



当 $N \gg 1$ 时, $N(N-1) \approx N^2$;
 ⇒ 随着 N 增大复乘、复加次数非线性增大。

N	2	4	8	16	32	64	128	256	512	1024	2048
N^2	4	16	64	256	1024	4096	16384	65536	262114	1048576	4194304
$N/2 \log_2 N$	1	4	12	32	80	192	448	1024	2304	5120	11264

同理：IDFT运算量与DFT相同。

减少运算量的基本途径

DFT定义中的相乘系数 W_N^{nk} 的性质是影响运算量的，可以看出， W_N^{nk} 有以下性质

$$W_N^{nk} = e^{-j\frac{2\pi}{N}nk}$$

共轭对称性 $(W_N^{nk})^* = W_N^{-nk} = W_N^{(N-n)k} = W_N^{n(N-k)}$

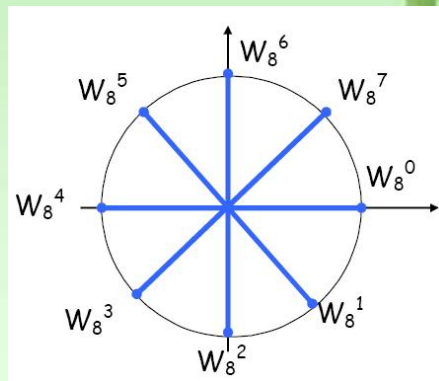
$$\downarrow$$
$$W_N^{Nk} \cdot W_N^{-nk}$$

$$\downarrow$$
$$W_N^{nN} \cdot W_N^{-nk}$$

周期性 $W_N^{nk} = W_N^{(N+n)k} = W_N^{n(N+k)}$

可约性 $W_N^{nk} = W_{mN}^{mnk} \quad W_N^{nk} = W_{N/m}^{nk/m}$

特殊点: $W_N^0 = 1 \quad W_N^{N/2} = -1 \quad W_N^{(k+N/2)} = -W_N^k$



$$DFT : \quad X(k) = DFT[x(n)] = \sum_{n=0}^{N-1} x(n)W_N^{kn}, 0 \leq k \leq N-1$$

$$\begin{bmatrix} X(0) \\ X(1) \\ X(2) \\ X(3) \end{bmatrix} = \begin{bmatrix} W_4^0 & W_4^0 & W_4^0 & W_4^0 \\ W_4^0 & W_4^{1 \times 1} & W_4^{1 \times 2} & W_4^{1 \times 3} \\ W_4^0 & W_4^{2 \times 1} & W_4^{2 \times 2} & W_4^{2 \times 3} \\ W_4^0 & W_4^{3 \times 1} & W_4^{3 \times 2} & W_4^{3 \times 3} \end{bmatrix} \begin{bmatrix} x(0) \\ x(1) \\ x(2) \\ x(3) \end{bmatrix}$$

$$W_4^0 = W_4^4 = 1 \quad W_4^1 = W_4^5$$

$$W_4^2 = W_4^6 = -1 = -W_4^0 \quad W_4^3 = -W_4^1$$

FFT基本思想

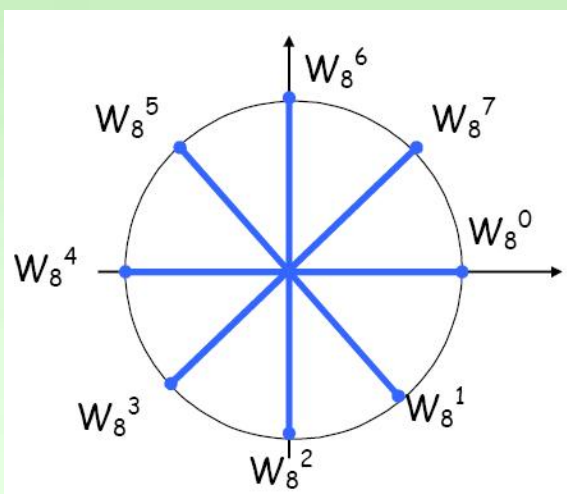
因为，DFT的运算次数 $\Leftrightarrow N^2$

若序列长度 $N \downarrow$ ，
则运算次数 $\downarrow$ 。

利用DFT系数的特性，合并
DFT运算中的同类项

长序列DFT $\longrightarrow$ 短序列DFT
从而减少其运算量

根据减少运算量的途径，巧妙地在时域或频域进行
不同的抽取分解与组合，可得到不同的快速算法。



■ 按时间抽取法

DIT: Decimation-In-Time

■ 按频率抽取法

DIF: Decimation-In-Frequency

4.2 按时间抽选的基-2FFT算法

(库利—图基算法)

$$DFT: \quad X(k) = DFT[x(n)] = \sum_{n=0}^{N-1} x(n)W_N^{kn}, 0 \leq k \leq N-1$$

$$\begin{bmatrix} X(0) \\ X(1) \\ \vdots \\ X(N-1) \end{bmatrix} = \begin{bmatrix} W_N^0 & W_N^0 & \cdots & W_N^0 \\ W_N^0 & W_N^{1 \times 1} & \cdots & W_N^{(N-1) \times 1} \\ \vdots & \vdots & & \vdots \\ W_N^0 & W_N^{1 \times (N-1)} & \cdots & W_N^{(N-1) \times (N-1)} \end{bmatrix} \begin{bmatrix} x(0) \\ x(1) \\ \vdots \\ x(N-1) \end{bmatrix}$$

$$DFT : \quad X(k) = DFT[x(n)] = \sum_{n=0}^{N-1} x(n) W_N^{kn}, 0 \leq k \leq N-1$$

$$\begin{bmatrix} X(0) \\ X(1) \\ X(2) \\ X(3) \end{bmatrix} = \begin{bmatrix} W_4^0 & W_4^0 & W_4^0 & W_4^0 \\ W_4^0 & W_4^{1 \times 1} & W_4^{1 \times 2} & W_4^{1 \times 3} \\ W_4^0 & W_4^{2 \times 1} & W_4^{2 \times 2} & W_4^{2 \times 3} \\ W_4^0 & W_4^{3 \times 1} & W_4^{3 \times 2} & W_4^{3 \times 3} \end{bmatrix} \begin{bmatrix} x(0) \\ x(1) \\ x(2) \\ x(3) \end{bmatrix}$$

$$W_4^0 = W_4^4 = 1 \quad W_4^1 = W_4^5$$

$$W_4^2 = W_4^6 = -1 = -W_4^0 \quad W_4^3 = -W_4^1$$

1、算法原理

N 为2的整数幂的FFT算法称**基-2FFT算法**。

对于基二算法，设序列点数 $N = 2^L$ ， L 为整数。若不满足，则补零。

将序列 $x(n)$ 按 n 的**奇偶**分成两组：

$$\begin{cases} x(2r) = x_1(r) \\ x(2r+1) = x_2(r) \end{cases}, \quad r = 0, 1, 2, \dots, N/2 - 1$$

将**N**点DFT定义式**分解**为两个长度为**N/2**的DFT

$$\begin{aligned} X(k) &= DFT[x(n)] = \sum_{n=0}^{N-1} x(n) W_N^{kn} \\ &= \underbrace{\sum_{r=0}^{N/2-1} x(2r) W_N^{2rk}}_{n \text{ 为偶}} + \underbrace{\sum_{r=0}^{N/2-1} x(2r+1) W_N^{(2r+1)k}}_{n \text{ 为奇}} \\ &= \underbrace{\sum_{r=0}^{N/2-1} x_1(r) W_{N/2}^{rk}}_{X_1(k)} + W_N^k \underbrace{\sum_{r=0}^{N/2-1} x_2(r) W_{N/2}^{rk}}_{X_2(k)} \end{aligned}$$

$$r, k = 0, 1, \dots, N/2 - 1$$

(这一步利用**可约性**: $W_N^{2rk} = W_{N/2}^{rk}$)

$$\text{记 } X(k) = X_1(k) + W_N^k X_2(k) \quad \dots\dots\dots(1)$$

$$\begin{aligned} W_N^{2rk} &= e^{-j \frac{2\pi}{N} 2rk} \\ &= e^{-j \frac{2\pi}{N/2} rk} \\ &= W_{N/2}^{rk} \end{aligned}$$

从此式就能方便地得到**N/2**个DFT系数，那么，还有**N/2**个DFT系数如何方便地得到呢？

$$X_1(k) = \sum_{r=0}^{N/2-1} x(2r)W_{N/2}^{rk} = \sum_{r=0}^{N/2-1} x_1(r)W_{N/2}^{rk}$$

只包含原序列的偶数点序列

$$X_2(k) = \sum_{r=0}^{N/2-1} x(2r+1)W_{N/2}^{rk} = \sum_{r=0}^{N/2-1} x_2(r)W_{N/2}^{rk}$$

只包含原序列的奇数点序列

再利用周期性求X(k)的后半部分

$$\because W_{N/2}^{r(N/2+k)} = W_{N/2}^{rk}$$



$$X_1\left(k + \frac{N}{2}\right) = X_1(k)$$

$$X_2\left(k + \frac{N}{2}\right) = X_2(k)$$

$X_1(k)$, $X_2(k)$ 为N/2点的DFT, 周期为N/2(由复指数序列的周期性可得)

$$X(k) = X_1(k) + W_N^k X_2(k)$$

$$X(k + \frac{N}{2}) = X_1(k + \frac{N}{2}) + W_N^{k + \frac{N}{2}} X_2(k + \frac{N}{2})$$

$$k = 0, 1, \dots, N/2 - 1$$

$$X_1(k + \frac{N}{2}) = X_1(k)$$

$$X_2(k + \frac{N}{2}) = X_2(k)$$

$$W_N^{k + \frac{N}{2}} = W_N^k \cdot W_N^{\frac{N}{2}} = -W_N^k$$

$$X(k) = X_1(k) + W_N^k X_2(k)$$

$$X(k + \frac{N}{2}) = X_1(k) - W_N^k X_2(k)$$

X(k)的前半部分k=0~N/2-1的组合方式

$$k = 0, 1, \dots, N/2 - 1$$

X(k)的后半部分k=N/2~N-1的组合方式

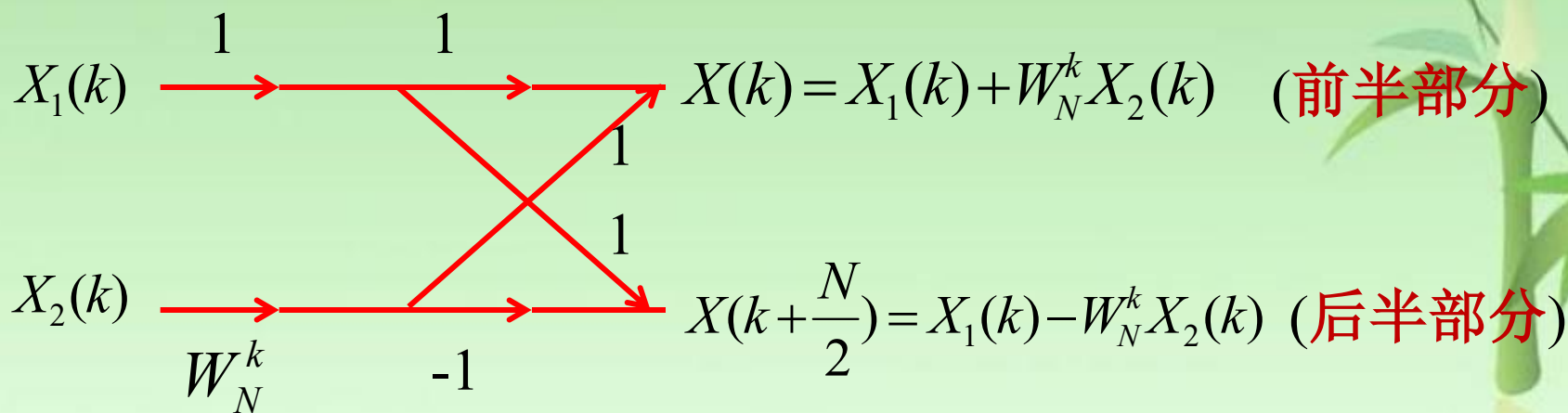
这样，只要求出0~N/2-1区间的所有X1(k)和X2(k)值，即可求出0~N-1区间内的所有的X(k)值，大大节省了运算。

2、蝶形运算

$$X(k) = X_1(k) + W_N^k X_2(k)$$

$$X(k + \frac{N}{2}) = X_1(k) - W_N^k X_2(k) \quad k = 0, 1, \dots, \frac{N}{2} - 1$$

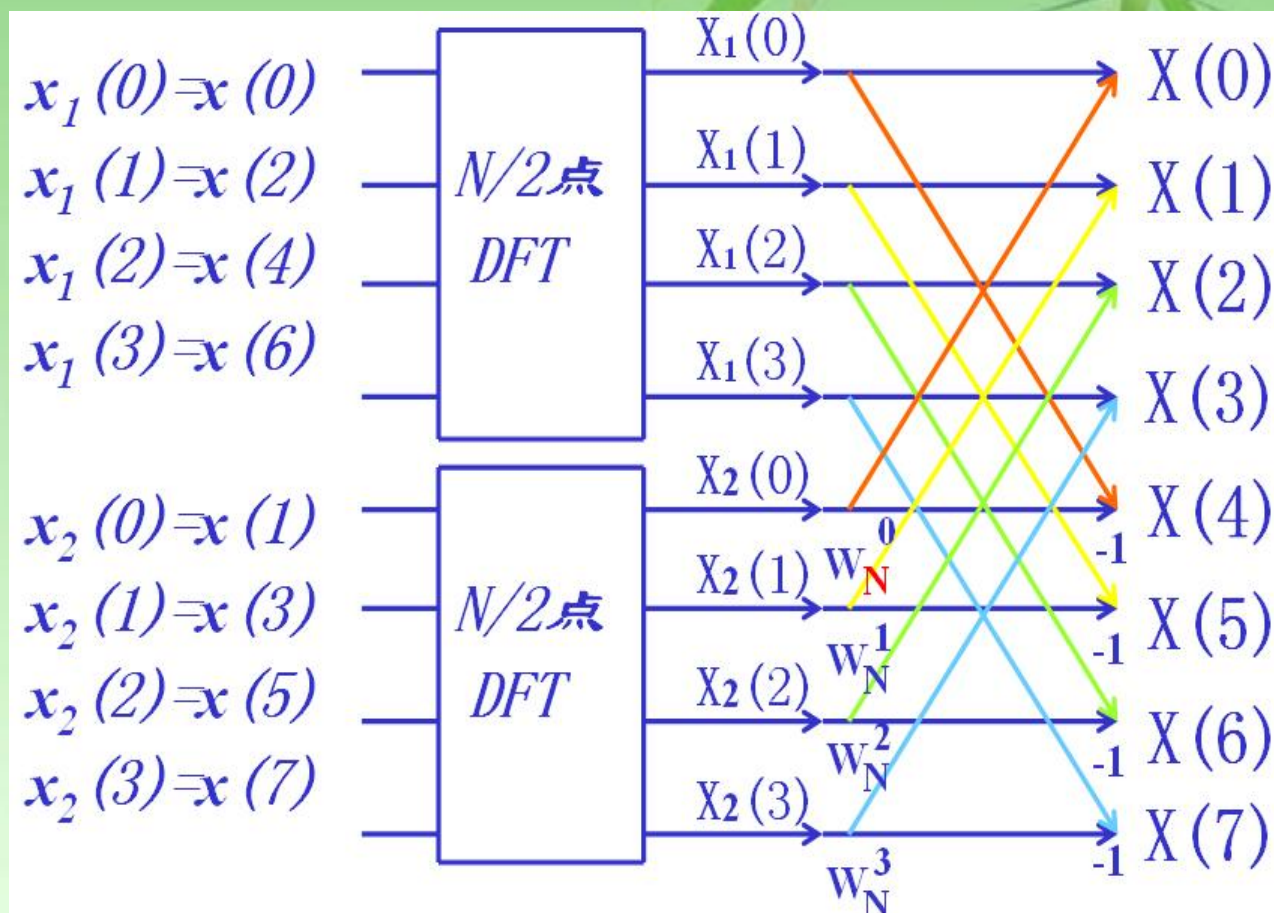
将上式表达的运算用一个专用“**蝶形**”运算流图表示。



支路上没有标出系数时，则该支路的传输系数为1

要完成一个蝶形运算，需**一次复数乘**和**两次复数加法**运算。

按时间抽取的FFT—8点FFT例子



按时间抽选，将一个N点DFT分解为两个N/2点DFT (N=8)

$$N=8=2^3$$

分解后的运算量:

	复数乘法	复数加法
一个 $N/2$ 点DFT	$(N/2)^2$	$N/2 (N/2 - 1)$
两个 $N/2$ 点DFT	$N^2/2$	$N(N/2 - 1)$
一个蝶形	1	2
$N/2$ 个蝶形	$N/2$	N
总计	$N^2/2 + N/2 \approx N^2/2$	$N(N/2 - 1) + N = N^2/2$

仅经过一次分解, 就使运算量减少近一半: $N^2 \rightarrow N^2/2$

进一步分解

由于 $N=2^L$, $N/2=2^{L-1}$ 仍为偶数, 因此, 两个 $N/2$ 点 DFT 又可同样进一步分解为 4 个 $N/4$ 点的 DFT。

与第一次分解相同, 将 $x_1(r)$ 按 r 的奇偶分解成两个 $N/4$ 长的子序列 $x_3(l)$ 和 $x_4(l)$, 即:

$$\begin{cases} x_1(2l) = x_3(l) \\ x_1(2l+1) = x_4(l) \end{cases} \quad l = 0, 1, \dots, N/4 - 1$$

$$\begin{aligned}
 X_1(k) &= \sum_{l=0}^{N/4-1} x_1(2l)W_{N/2}^{2lk} + \sum_{l=0}^{N/4-1} x_1(2l+1)W_{N/2}^{(2l+1)k} \\
 &= \sum_{l=0}^{N/4-1} x_1(2l)W_{N/2}^{2lk} + W_{N/2}^k \sum_{l=0}^{N/4-1} x_1(2l+1)W_{N/2}^{2lk} \\
 &= \sum_{l=0}^{N/4-1} x_3(l)W_{N/4}^{lk} + W_{N/2}^k \sum_{l=0}^{N/4-1} x_4(l)W_{N/4}^{lk} \\
 &= X_3(k) + W_{N/2}^k X_4(k)
 \end{aligned}$$

$$X_3(k) = \sum_{l=0}^{N/4-1} x_1(2l)W_{N/4}^{lk} = \sum_{l=0}^{N/4-1} x_3(l)W_{N/4}^{lk}$$

偶数点序列

$$X_4(k) = \sum_{l=0}^{N/4-1} x_1(2l+1)W_{N/4}^{lk} = \sum_{l=0}^{N/4-1} x_4(l)W_{N/4}^{lk}$$

奇数点序列

$X_3(k)$, $X_4(k)$ 为N/4点的DFT, 周期为N/4

$$X_3(k) = X_3(k + \frac{N}{4})$$

$$X_4(k) = X_4(k + \frac{N}{4})$$

$$\begin{cases} X_1(k) = X_3(k) + W_{N/2}^k X_4(k) \\ X_1(k + \frac{N}{4}) = X_3(k) - W_{N/2}^k X_4(k) \end{cases} \quad k = 0, 1, \dots, \frac{N}{4} - 1$$

“蝶形” 信流图表示

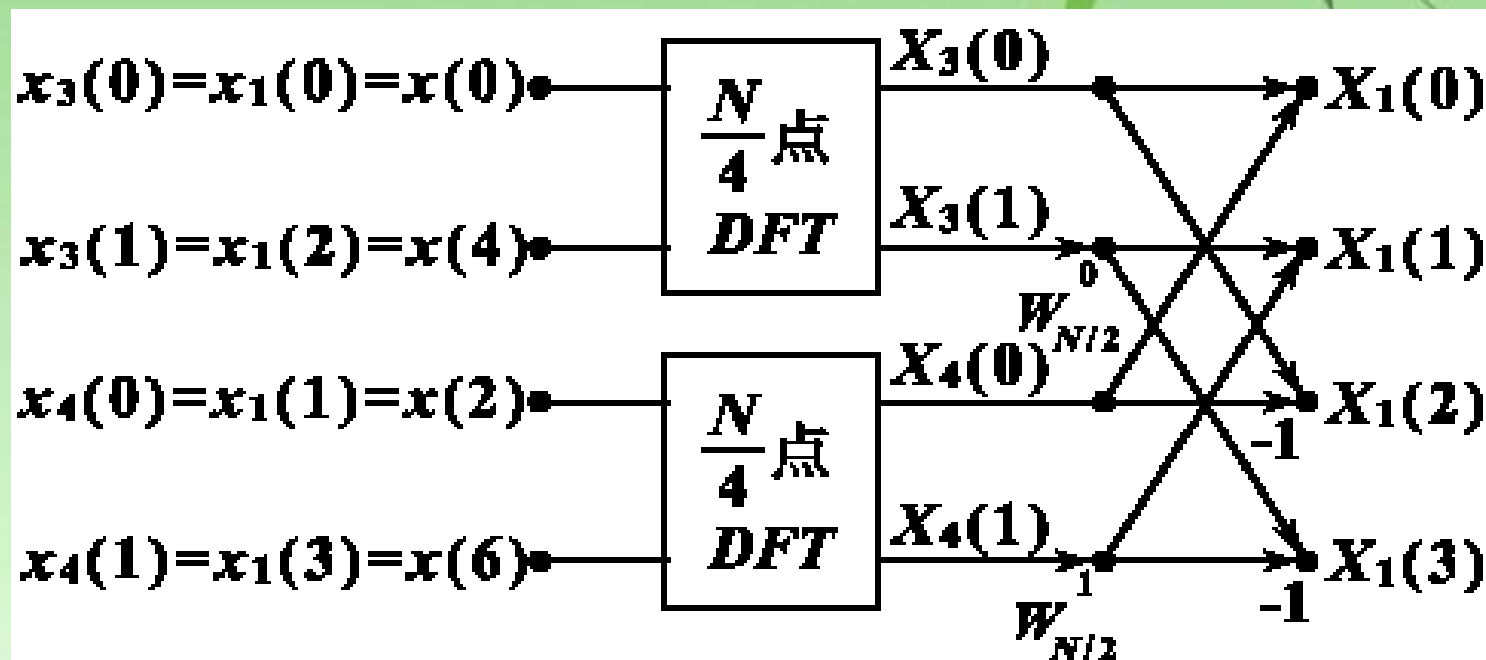


图4-3 由两个 $N/4$ 点DFT组合成一个 $N/2$ 点DFT

用同样的方法，将 $x_2(r)$ 按 r 的奇偶分解，可得：

同理：

$$\begin{cases} X_2(k) = X_5(k) + W_{N/2}^k X_6(k) \\ X_2(k + \frac{N}{4}) = X_5(k) - W_{N/2}^k X_6(k) \end{cases} \quad k = 0, 1, \dots, \frac{N}{4} - 1$$

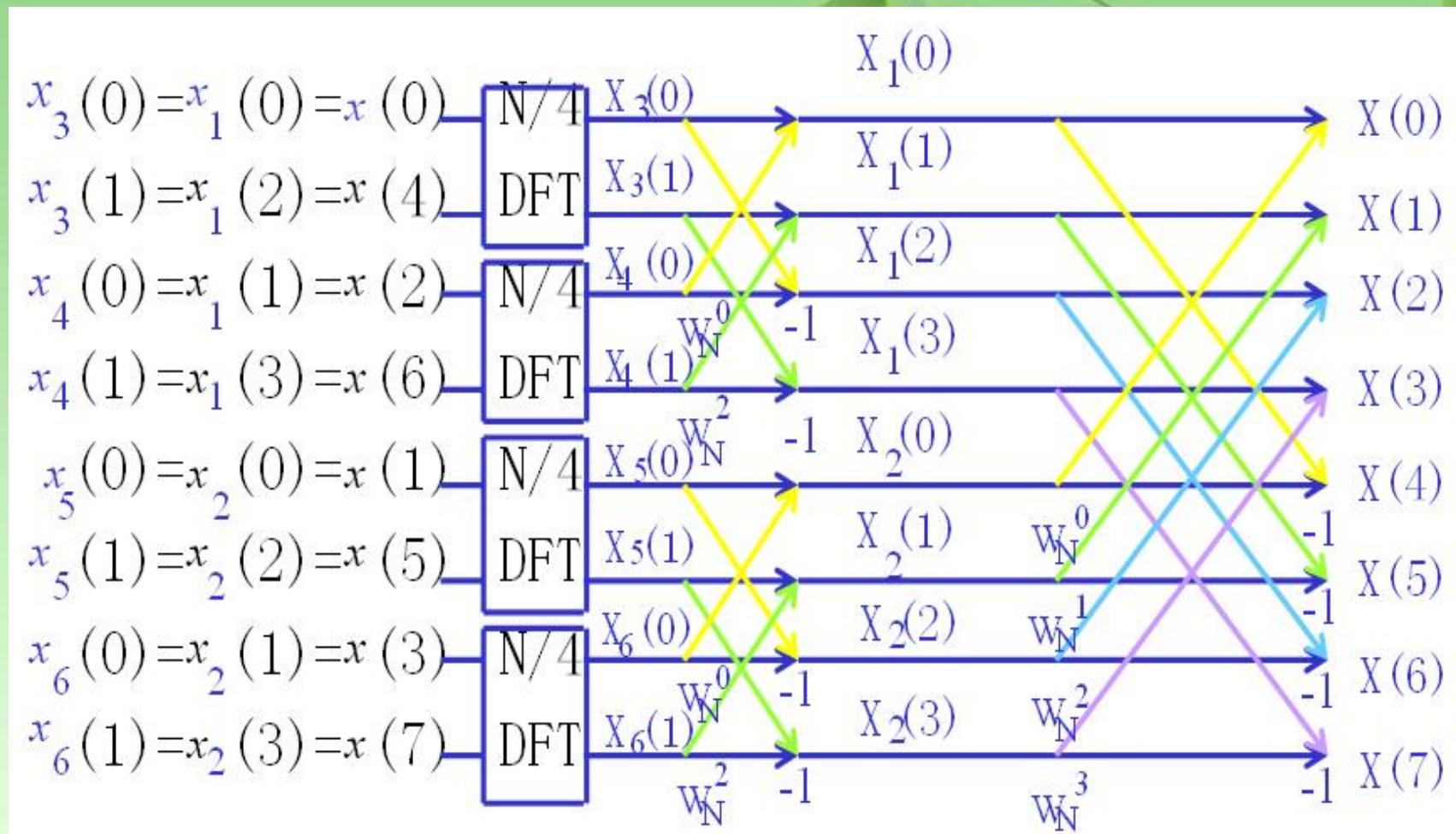
其中：

$$X_5(k) = DFT[x_5(l)] = DFT[x_2(2l)] \quad l = 0, 1, \dots, N/4 - 1$$

$$X_6(k) = DFT[x_6(l)] = DFT[x_2(2l + 1)]$$

$$\text{统一系数： } W_{N/2}^k \rightarrow W_N^{2k}$$

$N=8$ 时, $x_3(l)$ 和 $x_4(l)$ 已是两个**长度为2**的子序列:



按时间抽选, 由两个 $N/4$ 点DFT组合成一个 $N/2$ 点DFT ($N=8$)

同上, 利用**4个** $N/4$ 点的DFT及**两级蝶形**组合运算来计算 N 点DFT, 比只利用一次分解蝶形组合的方式的计算量又**减少了近一半**

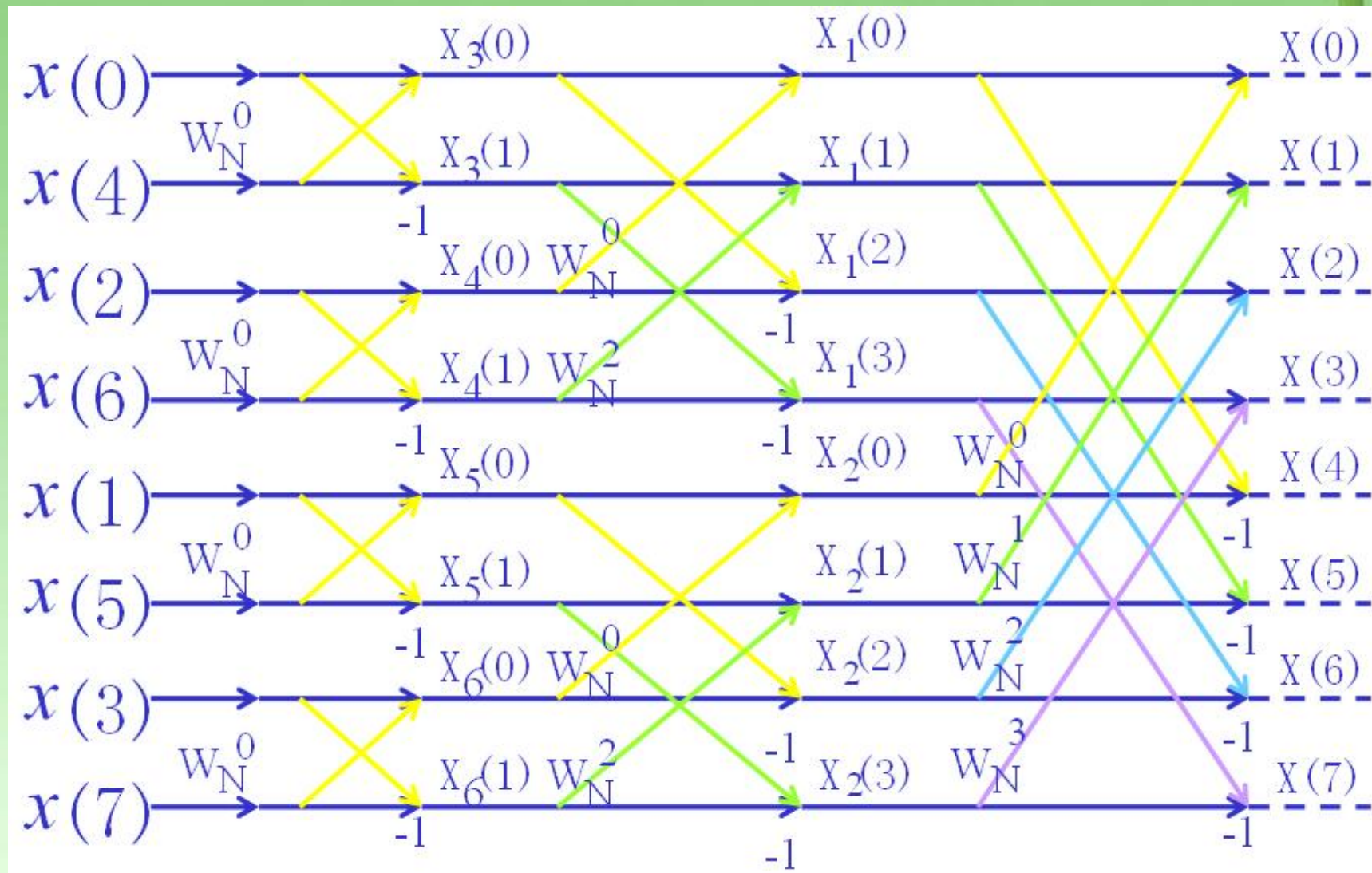
这样逐级分解，直到2点DFT当 $N=8$ 时，即分解到 $X_3(k)$, $X_4(k)$, $X_5(k)$, $X_6(k)$, $k=0, 1$

$$X_3(k) = \sum_{l=0}^{N/4-1} x_3(l) W_{N/4}^{lk} = \sum_{l=0}^1 x_3(l) W_{N/4}^{lk} \quad k=0,1$$

$$\begin{cases} X_3(0) = x_3(0)W_2^0 + W_2^0 x_3(1) = x(0) + W_N^0 x(4) \\ X_3(1) = x_3(0)W_2^0 + W_2^1 x_3(1) = x(0) - W_N^0 x(4) \end{cases}$$

$$X_4(k) = \sum_{l=0}^{N/4-1} x_4(l) W_{N/4}^{lk} = \sum_{l=0}^1 x_4(l) W_{N/4}^{lk} \quad k=0,1$$

$$\begin{cases} X_4(0) = x_4(0)W_2^0 + W_2^0 x_4(1) = x(2) + W_N^0 x(6) \\ X_4(1) = x_4(0)W_2^0 + W_2^1 x_4(1) = x(2) - W_N^0 x(6) \end{cases}$$



经过 $L-1$ 次分解，将 N 点DFT分解成了 $N/2$ 个2点的DFT，因此 $N=2^L$ 时，FFT总共需要 **L 级**运算，每级 **$N/2$ 个蝶形**。

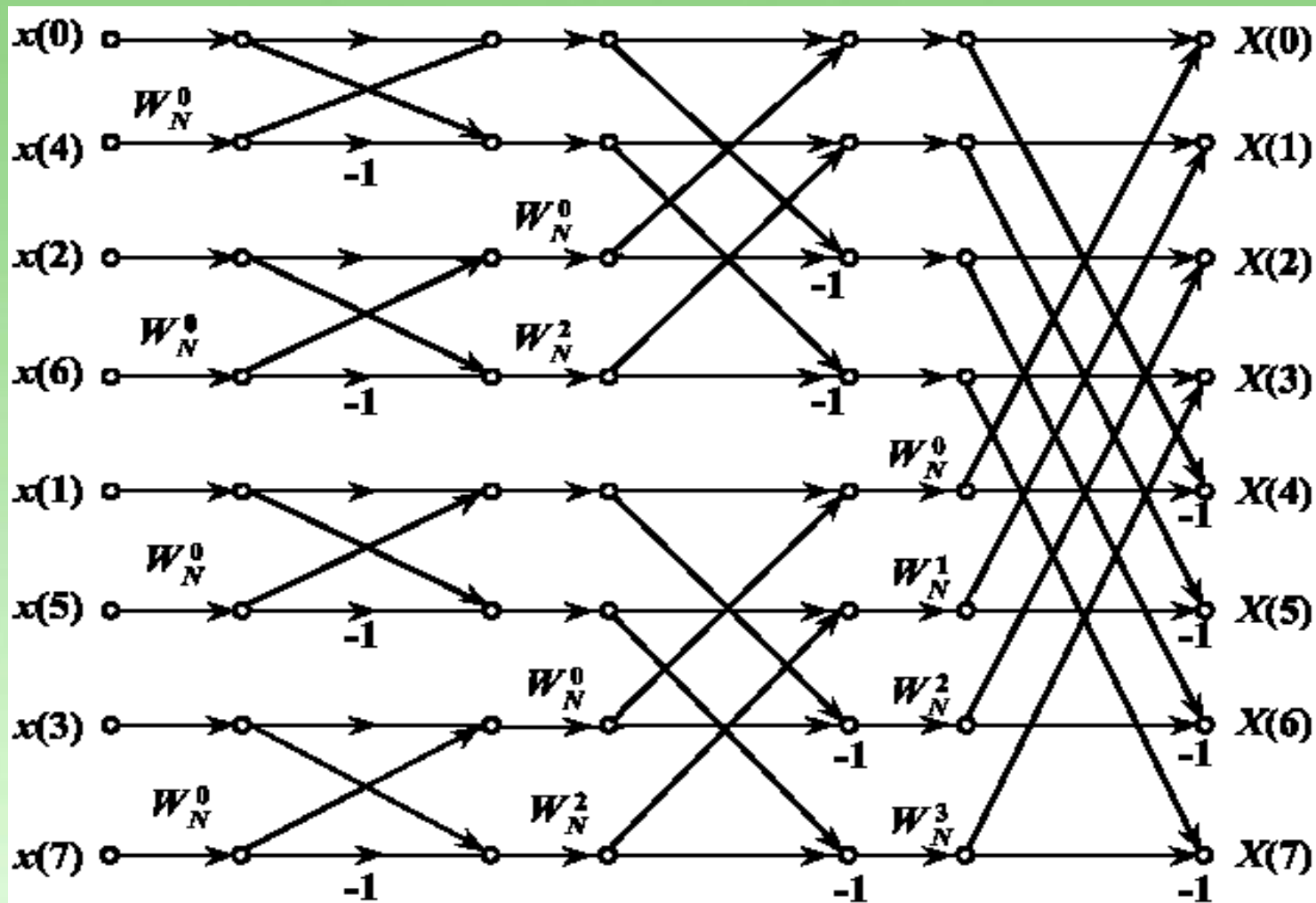


图4-5 $N=8$ 按时间抽选法FFT运算流图

经过 $L-1$ 次分解，将 N 点DFT分解成了 $N/2$ 个2点的DFT，因此 $N=2^L$ 时，FFT总共需要 L 级运算，每级 $N/2$ 个蝶形。

2、运算量

这种方法的每一步分解都是按输入序列在时域上的次序是属于偶数还是奇数来抽取的，所以称为“按时间抽取法” (DIT)。

设序列点数 $N = 2^L$ ， L 为整数。若不满足，则补零。补零后，时域点数增加，但有效数据不变，故频谱 $X(e^{j\omega})$ 不变，只是频谱的抽样点增加，因而抽样点位置改变。

$N=2^L$ 时，共有 $L=\log_2 N$ 级运算；每一级有 $N/2$ 个蝶形运算，每个蝶形运算有一次复乘,两次复加。

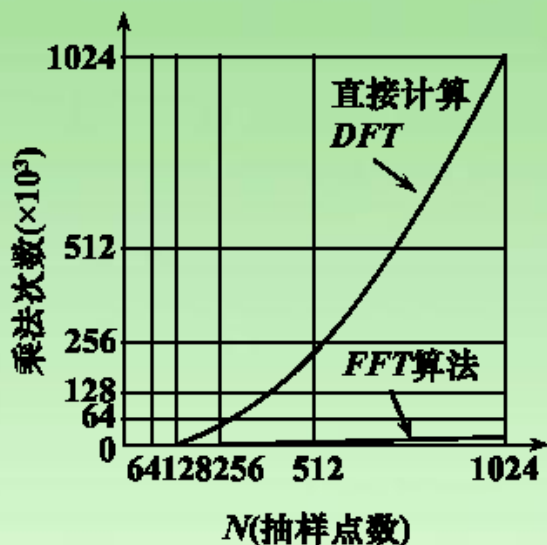
因此, N 点的FFT的运算量为

$$\text{复乘: } m_F = \frac{N}{2} L = \frac{N}{2} \log_2 N$$

$$\text{复加: } a_F = NL = N \log_2 N$$

比较DFT

$$\frac{m_F(DFT)}{m_F(FFT)} = \frac{N^2}{\frac{N}{2} \log_2 N} = \frac{2N}{\log_2 N}$$



可以直观看出，当点数 N 越大时，FFT算法的运算效率更高。 $N=2^8$ 时,比值为64。 $N=2^{10}=1024$ 时,比值为204.8。

在实际运算中，FFT的运算量还要减少，因为 $W_N^N=1$ ， $W_N^0=1$ ， $W_N^{N/2}=-1$ ， $W_N^{N/4}=-j$ ，此时不需进行复数乘法。

图4-6 直接计算DFT与FFT算法所需乘法次数的比较

N	2	4	8	16	32	64	128	256	512	1024	2048
N^2	4	16	64	256	1024	4096	16384	65536	262114	1048576	4194304
$N/2 \log_2 N$	1	4	12	32	80	192	448	1024	2304	5120	11264

3、算法特点——原位/同址计算

DIT-FFT的运算过程很有规律。 $N=2^L$ 点的FFT共进行**L级**运算，每级由 **$N/2$ 个蝶形**运算组成。

同一级中，每个蝶形的两个输入数据只对计算本蝶形有用，而且每个蝶形的输入、输出数据结点又同在同一条水平线上，即计算完一个蝶形后，所得输出数据可立即存入**原**输入数据所占用的存贮单元。

这种利用同一存贮单元存贮蝶形计算输入、输出数据的方法称为**原位(址)计算**。

原位计算可节省大量内存，使设备成本降低。

这是FFT算法的一大优点。

共有 $L=\log_2 N$ 级蝶形运算

输入数据、中间运算结果和最后输出均用同一存储器。**节省存储单元**

$$\begin{cases} X_m(k) = X_{m-1}(k) + X_{m-1}(j)W_N^r \\ X_m(j) = X_{m-1}(k) - X_{m-1}(j)W_N^r \end{cases}$$

m 表示第 m 级迭代, k, j 表示数据所在的行数

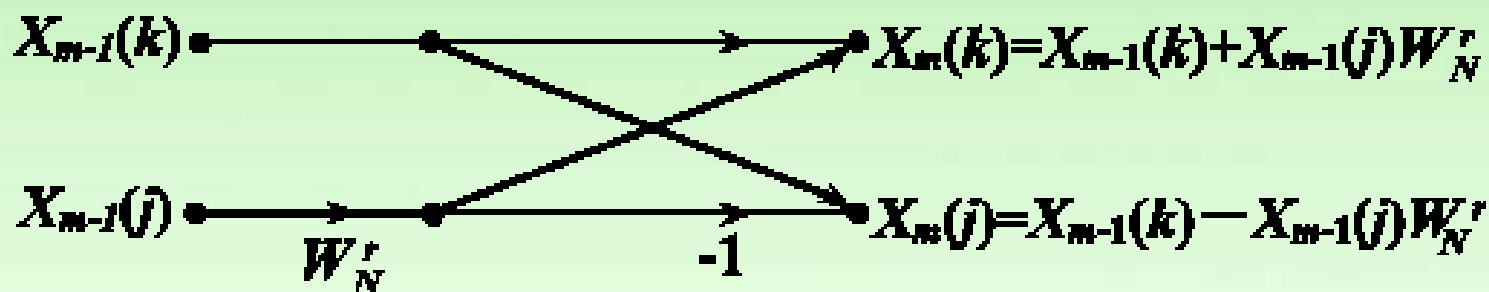
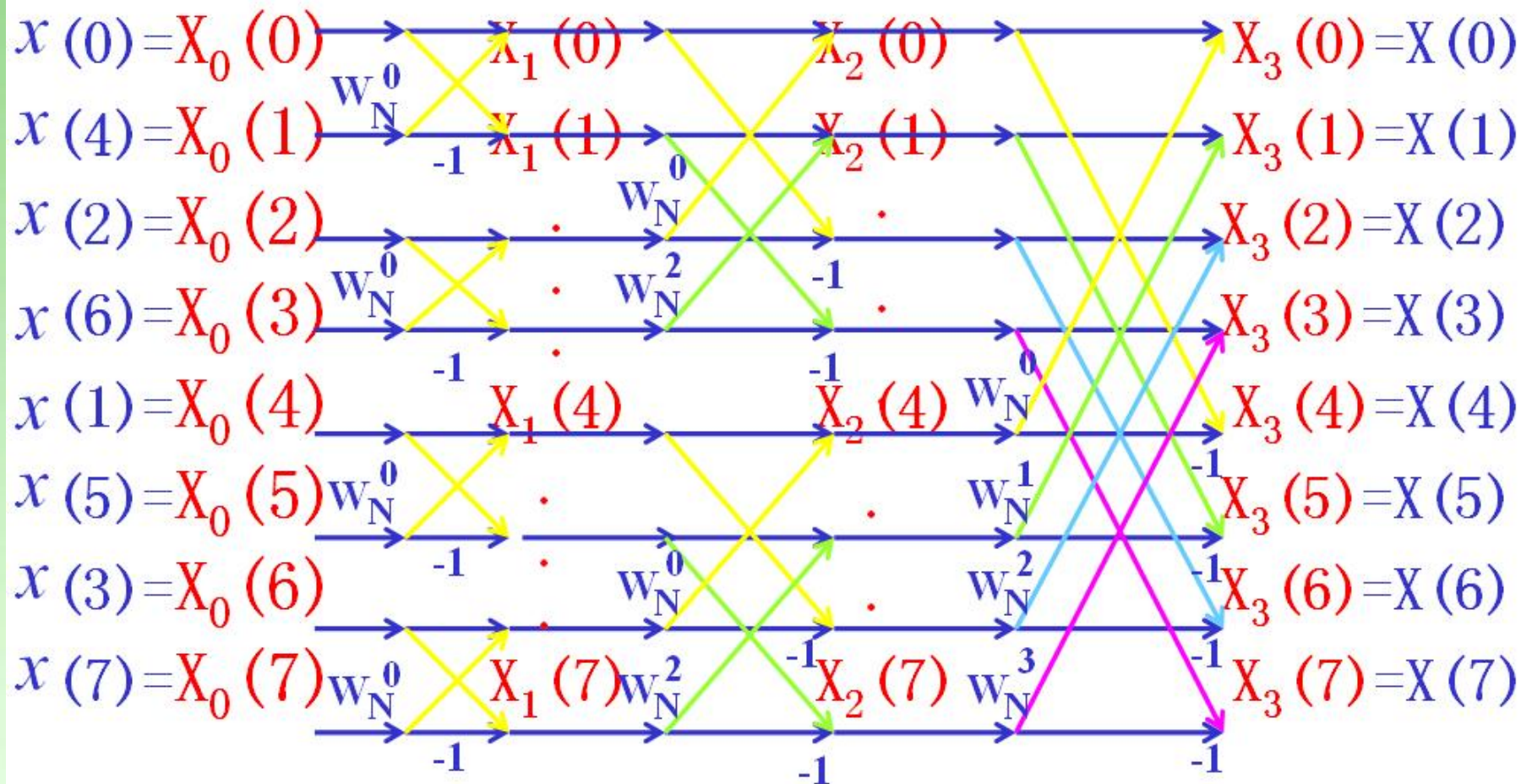


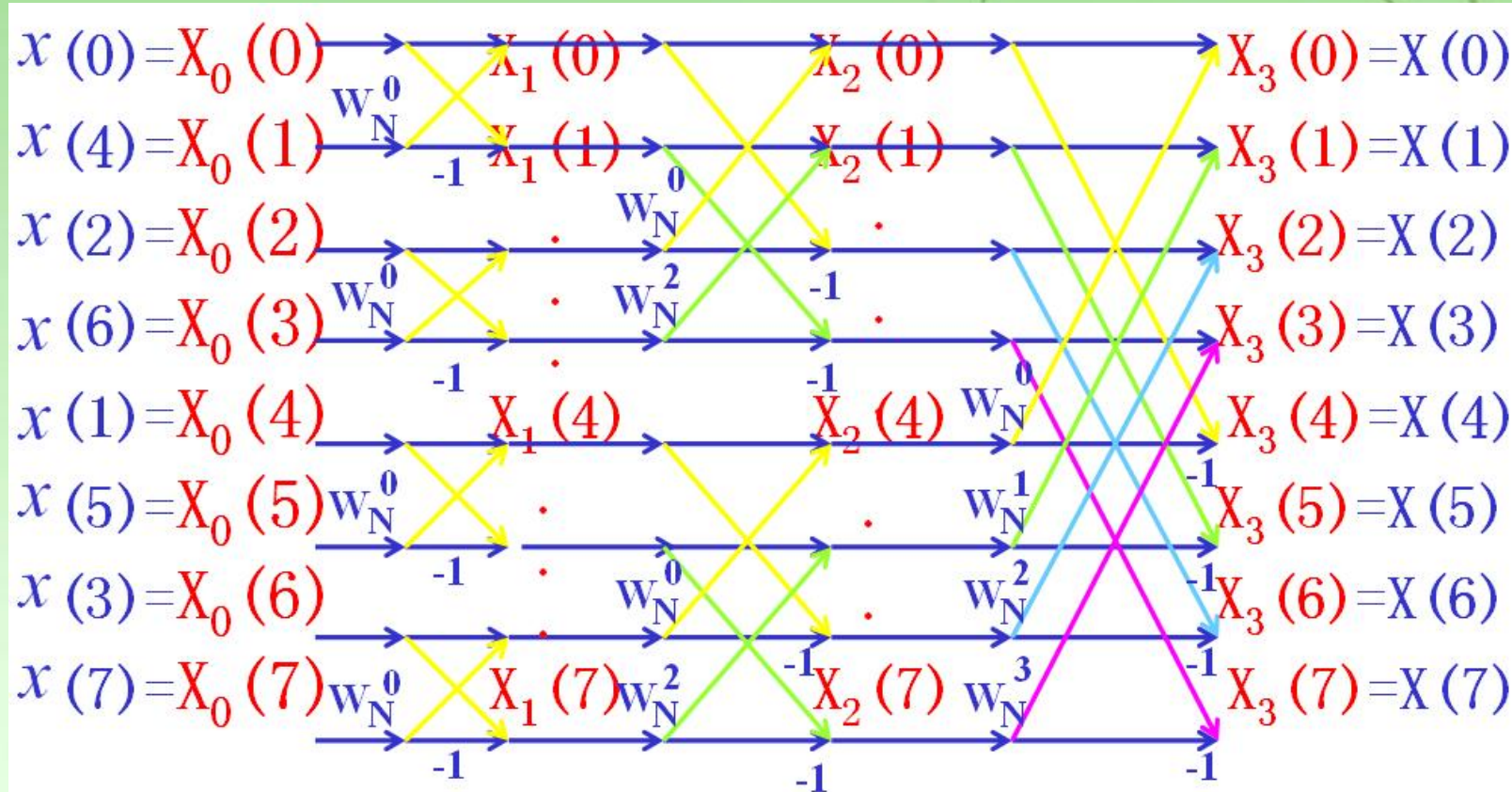
图 4-7 按时间抽选蝶形运算结构



输入数据、中间运算结果和最后输出均用同一存储器。

3、算法特点——倒序位

输出 $X(k)$ 按正常顺序排列在存储单元,而输入是按顺序 $x(0)$, $x(4)$, ... $x(7)$ 排列的, 这种顺序称作**倒位序**,即二进制数倒位。



DIT-FFT算法的输入序列的排序看起来似乎很乱，但实际排序是很有规律的。

实现按时间抽取FFT算法的**关键**是将输入数据排列成满足连续运用**奇偶**分解所需的次序。由于输入序列按时间序位的**奇偶**抽取，故输入序列是混序的，为此需要先进行混序处理。

混序规律： $x(n)$ 按 n 位置进行码位（二进制）倒置规律输入，而非自然排序，即得到混序排列。所以称为**位倒序处理**。

$N=2^L$ ，原输入序列的顺序表示为： $\mathbf{n}=(n_{L-1}n_{L-2}\dots n_1n_0)_2$
(L 位二进制数表达时序)

对输入数据次序的变化可根据一个简单的位对换规则进行（称为倒位序）： $\mathbf{n}'=(n_0n_1\dots n_{L-2}n_{L-1})_2$

当把输入数据进行了重新排序，则输出结果是正确的次序。例： $n=2^3$ ，输入序“6”重新排序后为“3”，因为 $6=110_2$ 倒位序后 $n'=011_2=3$ 。

•倒位序的实现

$$n = (n_2 n_1 n_0)_2$$

$$\hat{n} = (n_0 n_1 n_2)_2$$

自然顺序 n	二进制 $n_2 n_1 n_0$	倒位序二进制 $n_0 n_1 n_2$	倒位顺序 $\hat{n}$
0	0 0 0	0 0 0	0
1	0 0 1	1 0 0	4
2	0 1 0	0 1 0	2
3	0 1 1	1 1 0	6
4	1 0 0	0 0 1	1
5	1 0 1	1 0 1	5
6	1 1 0	0 1 1	3
7	1 1 1	1 1 1	7

•倒位序

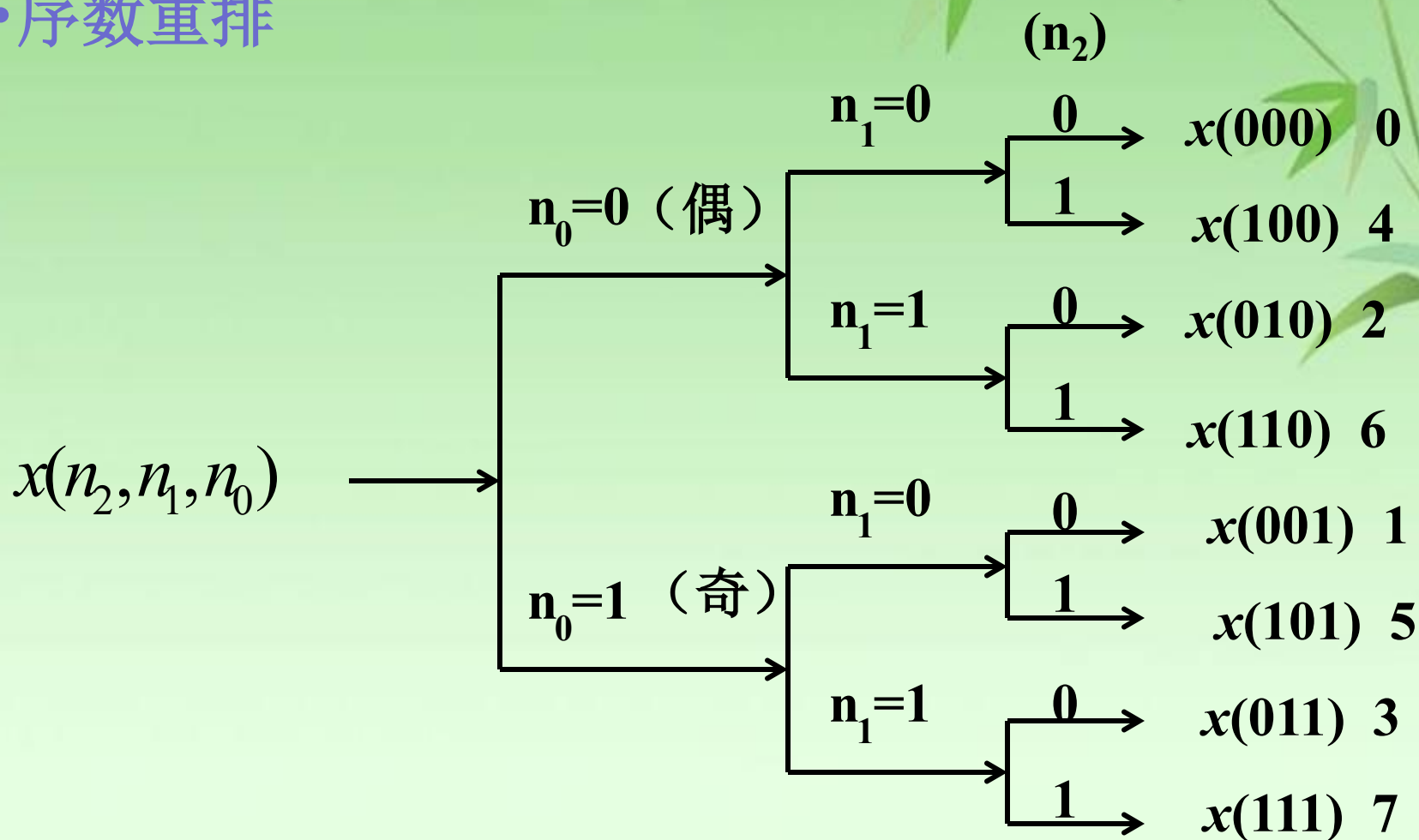
n_0	n_1	n_2
0	0	0
		1
	1	0
1	0	1
		0
	1	0
		1

$$x(n) \quad n = (n_2 n_1 n_0)_2$$

倒位序 $\hat{n} = (n_0 n_1 n_2)_2$		自然序 $n = (n_2 n_1 n_0)_2$	
000	0	0	000
100	4	1	001
010	2	2	010
110	6	3	011
001	1	4	100
101	5	5	101
011	3	6	110
111	7	7	111

输入倒序位，输出自然序位(正序)

• 序数重排



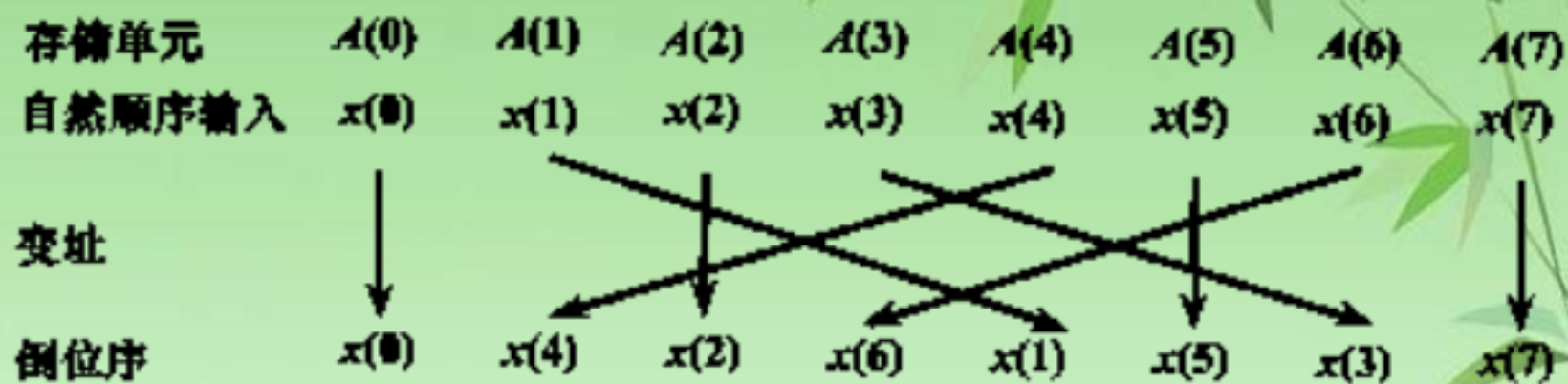


图4-9 倒位序的变址处理

例 计算 $x(n) = n^2$, $n = 0, 1, 2, \dots, 31$ 。计算 $N = 32$ 点FFT。用时间抽取输入倒序算法，问倒序前寄存器的数 $x(13)$ 和倒序后 $x(13)$ 的数据值？

解：倒序前 $x(13) = 13^2 = 169$

倒序 $(13)_{10} = (01101)_2$ $N = 32 = 2^5$

倒序为 $(10110)_2 = (22)_{10}$

倒序后 $x(13) = 22^2 = 484$

3、算法特点——蝶形运算

对 $N = 2^L$ 点FFT，输入倒位序，输出自然序，
第 m 级运算每个蝶形的两节点（信号）距离（码地址）
为 2^{m-1} ，

$$m = 1, 2, \dots, L \quad (N = 2^L)$$

$$k = 0, 1, 2, \dots, 2^{m-1} - 1$$

第 m 级运算：

$$\begin{cases} X_m(k) = X_{m-1}(k) + X_{m-1}(k + 2^{m-1})W_N^r \\ X_m(k + 2^{m-1}) = X_{m-1}(k) - X_{m-1}(k + 2^{m-1})W_N^r \end{cases}$$

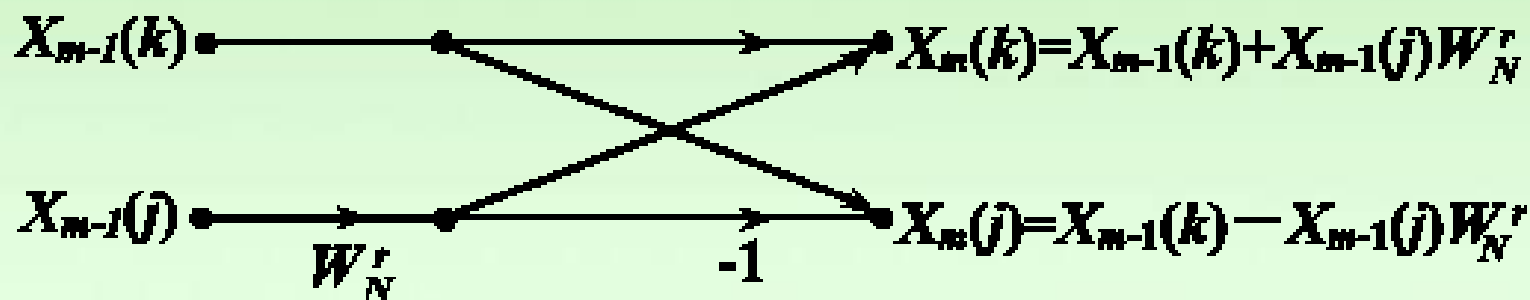


图 4-7 按时间抽选蝶形运算结构

- 蝶形类型随迭代次数成倍增加

W_N^r 第 m 级运算共有 2^{m-1} 蝶形运算系数/蝶形类型和取数间隔

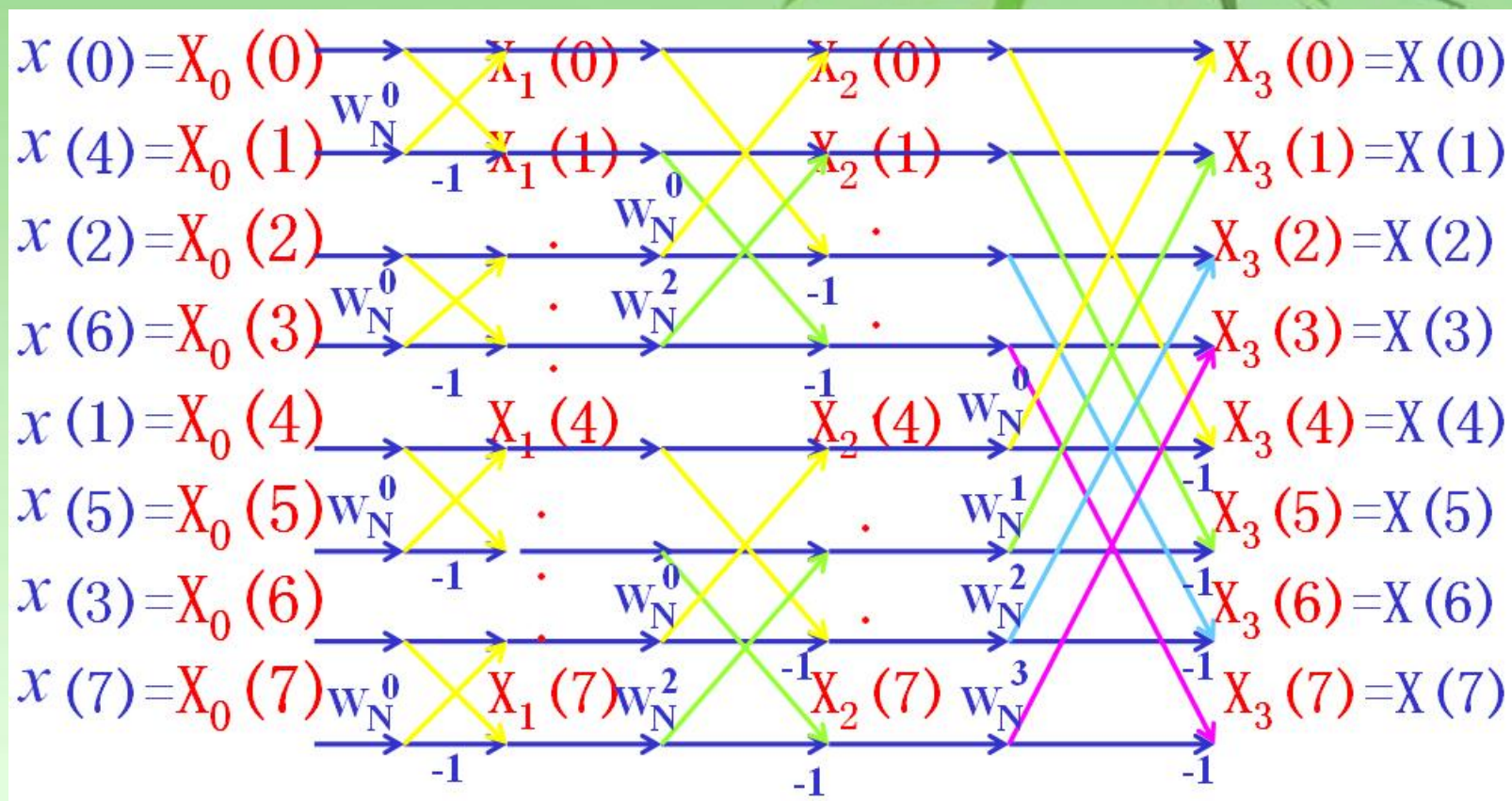
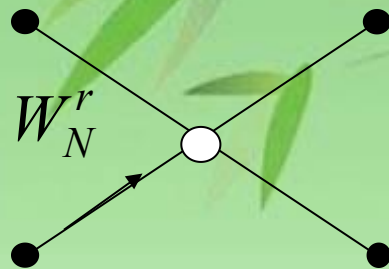


图4-5 $N=8$ 按时间抽选法FFT运算流图

每迭代一次，蝶形类型增加一倍，数据点间隔也增大一倍。

W_N^r 的确定:

第 m 级的 r 取值:



- 蝶形运算两节点的第一个节点为 k 值，表示成 L 位二进制数，左移 $L-m$ 位，把右边空出的位置补零，结果为 r 的二进制数。

$$r = (k)_2 \cdot 2^{L-m}$$

$$r = k \cdot 2^{L-m} \quad k = 0, 1, 2, \dots, 2^{m-1} - 1$$

旋转因子的变化规律

如上所述， N 点DIT-FFT运算流图中，每级都有 $N/2$ 个蝶形。每个蝶形都要乘以因子 W_N^r ，称其为**旋转因子**， r 为旋转因子的指数。但各级的旋转因子和循环方式都有所不同。

用 m 表示从左到右的运算级数($m=1, 2, \dots, L$)。观察FFT运算示意图不难发现，第 m 级共有 2^{m-1} 个不同的旋转因子。

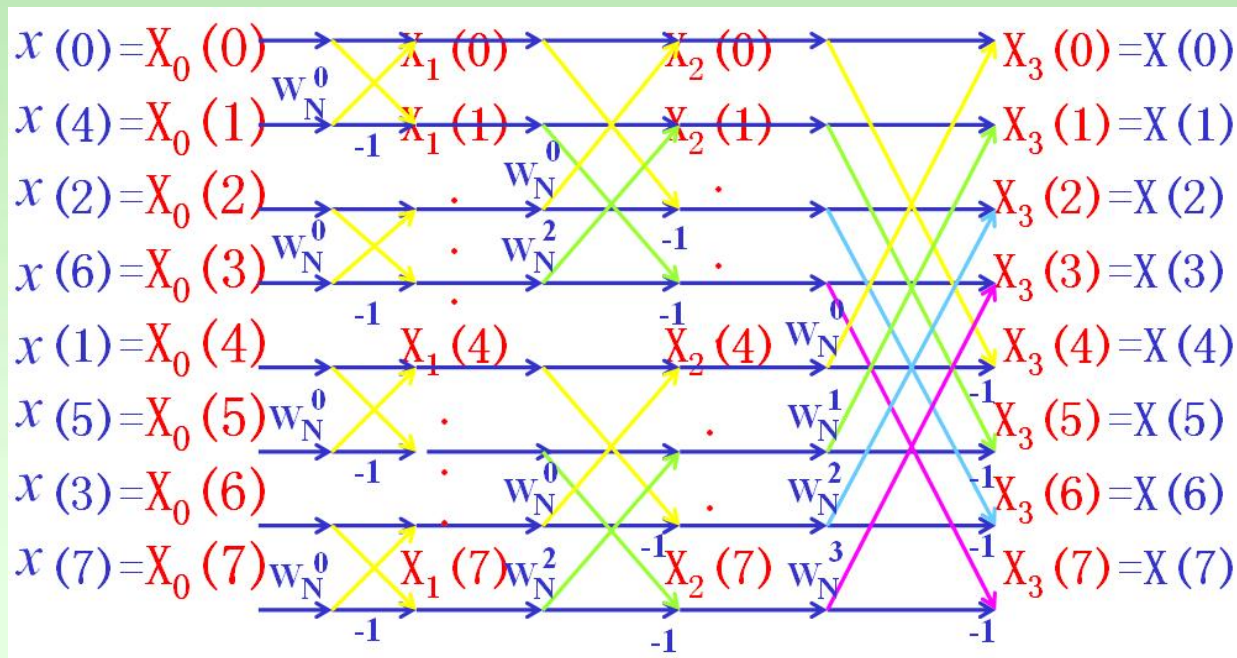
$$m = 1 \text{ 时, } W_N^r = W_{N/4}^k = W_{2^m}^k \quad k = 0$$

$$m = 2 \text{ 时, } W_N^r = W_{N/2}^k = W_{2^m}^k \quad k = 0, 1$$

$$m = 3 \text{ 时, } W_N^r = W_N^k = W_{2^m}^k \quad k = 0, 1, 2, 3$$

对 $N=2^L$ 的一般情况，第 m 级的旋转因子为

$$W_N^r = W_{2^m}^k \quad k = 0, 1, 2, \dots, 2^{m-1} - 1$$



因为 $2^m = 2^L 2^{m-L} = N \cdot 2^{m-L}$

所以 $W_N^r = W_{2^m}^k = W_{N \cdot 2^{m-L}}^k = W_N^{k \cdot 2^{L-m}} \quad k = 0, 1, 2, \dots, 2^{m-1} - 1$

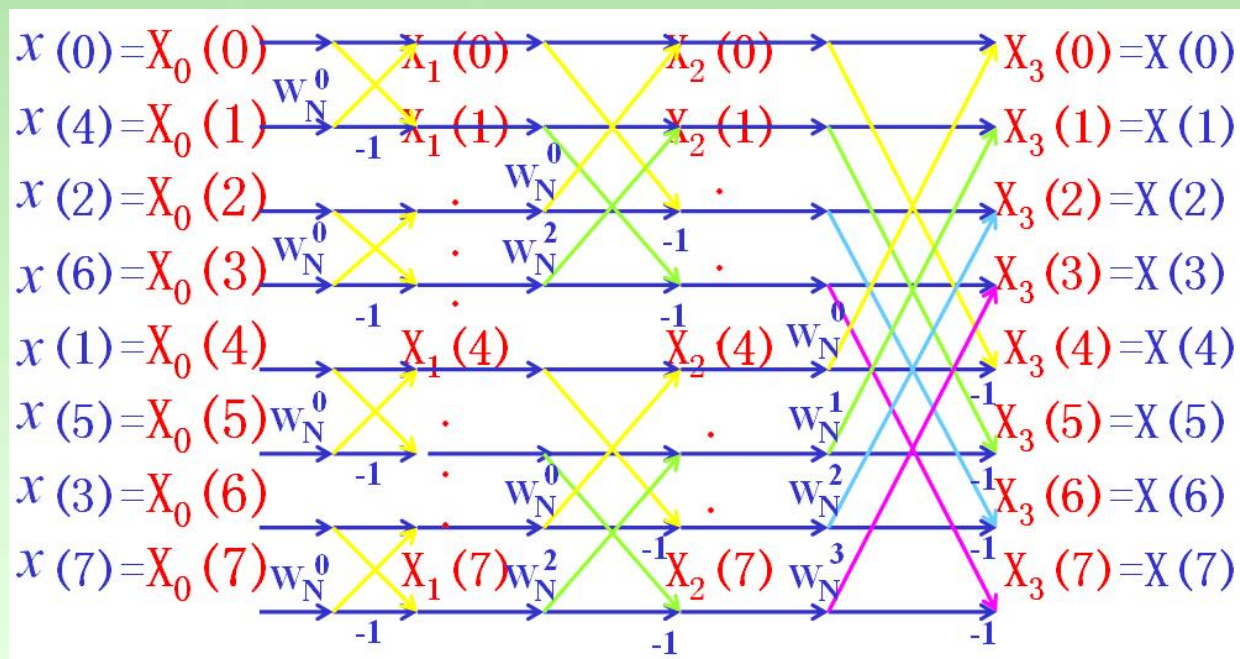
$$r = k \cdot 2^{L-m}$$

这样，就可按上式确定第 m 级运算的旋转因子

■ 存储单元

输入序列 $x(n)$: N 个存储单元

系数 W_N^r : $N/2$ 个存储单元



DIT算法的其他形式流图

- 输入倒位序输出自然序
- 输入自然序输出倒位序
- 输入输出均自然序
- 相同几何形状
 - 输入倒位序输出自然序
 - 输入自然序输出倒位序

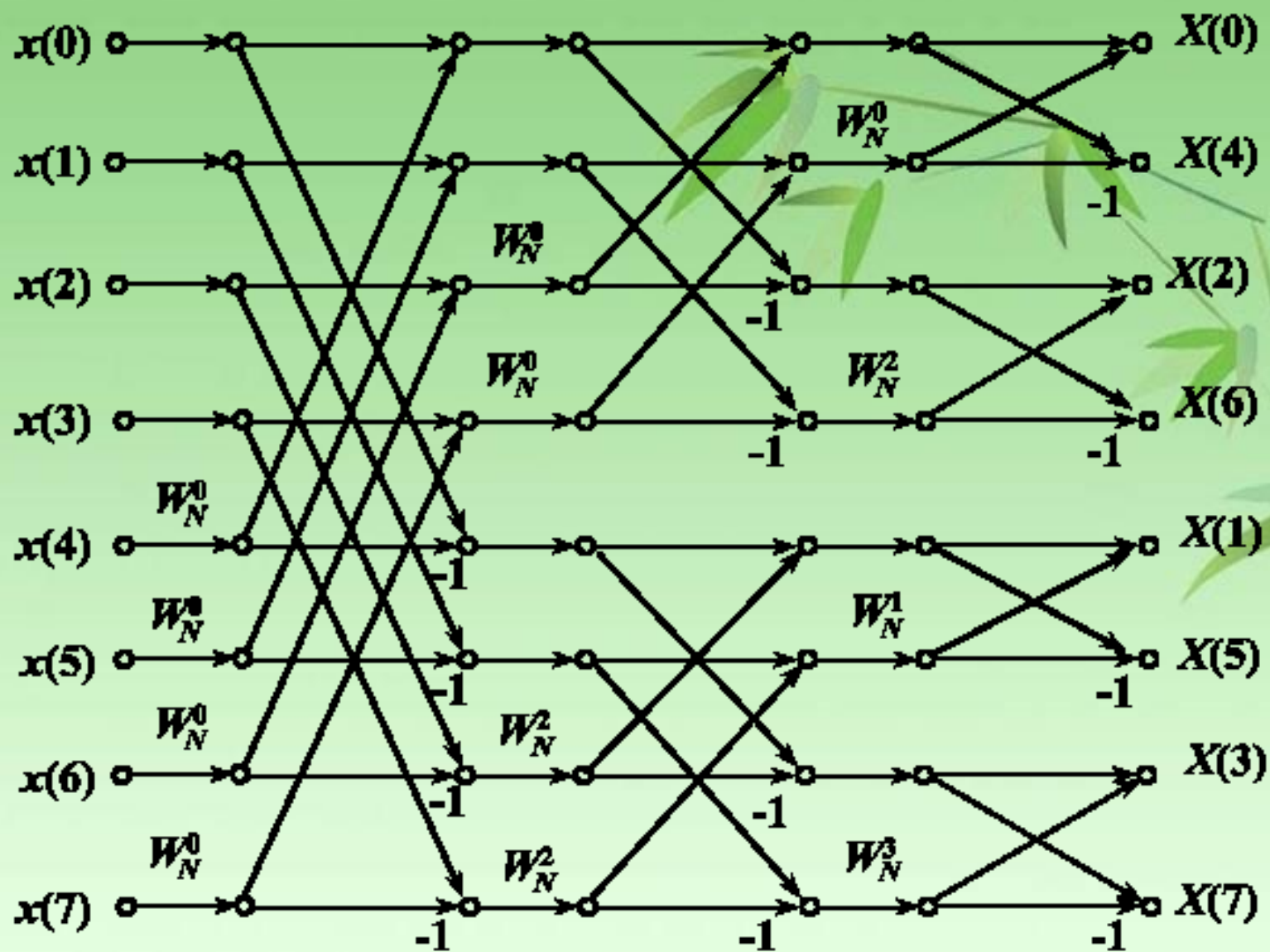


图4-10 按时间抽选，输入自然顺序，输出倒位序的FFT图

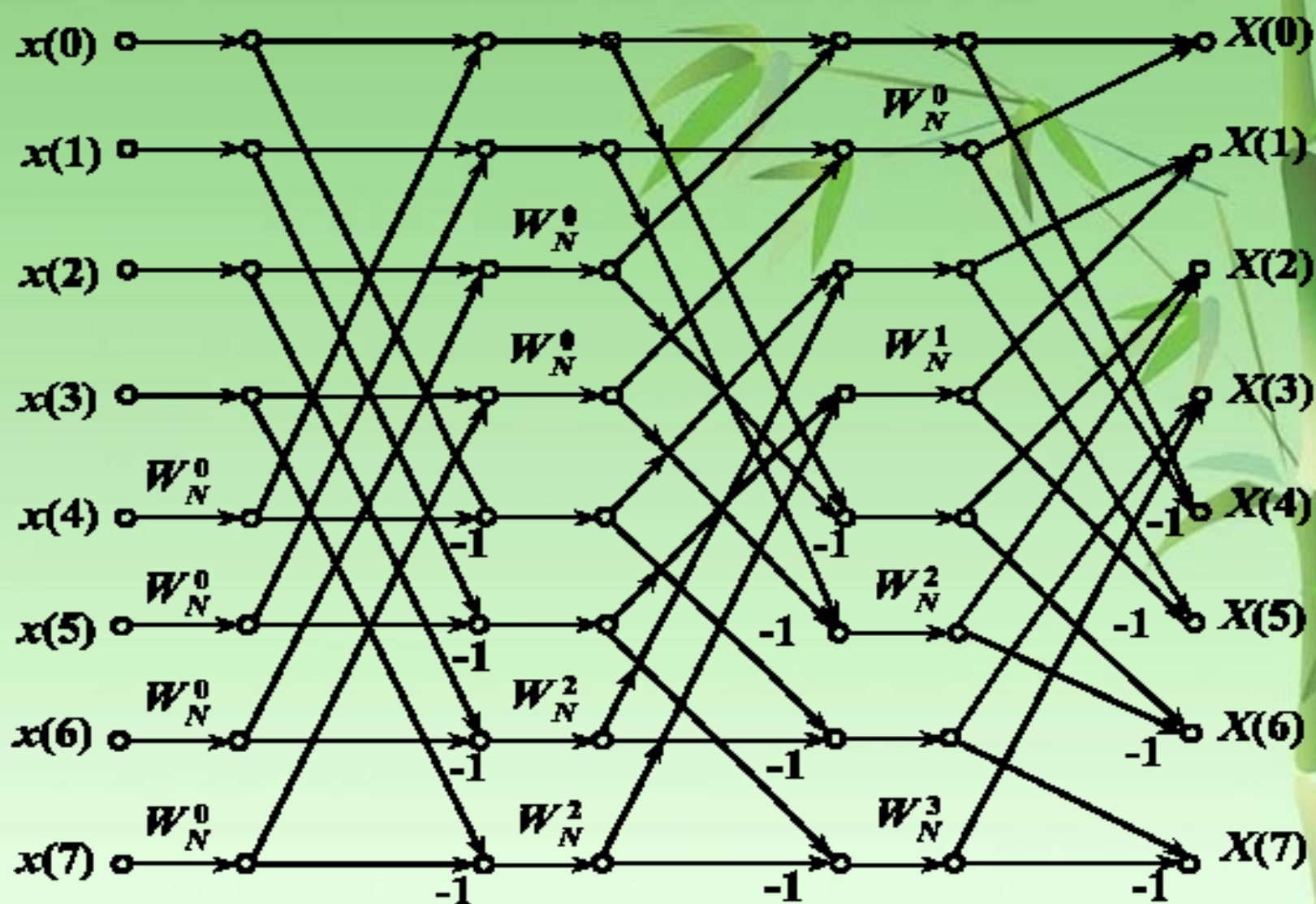


图4-11 按时间抽选,输入输出皆为自然顺序的FFT流图

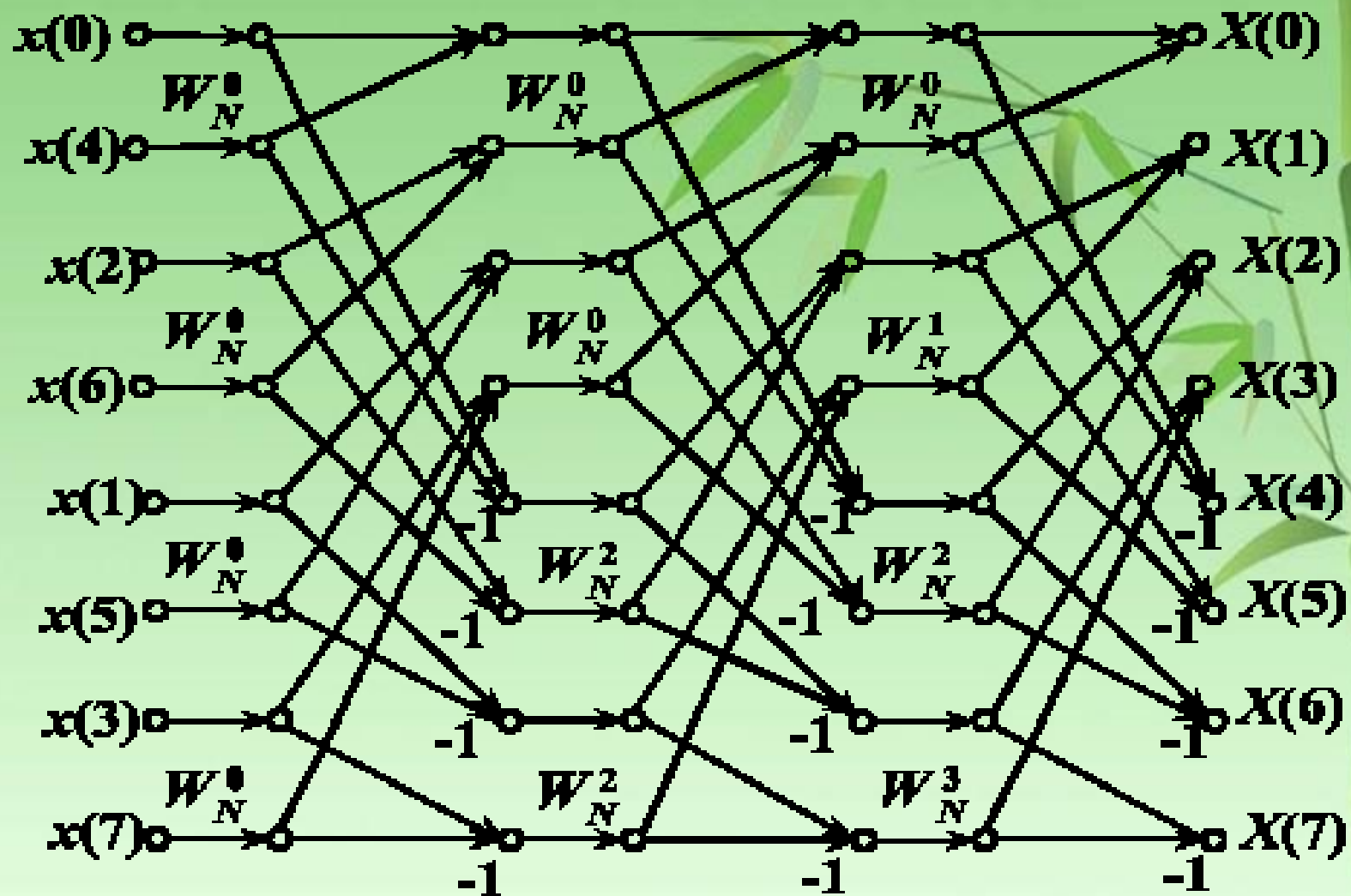


图 4-12 按时间抽选，各级具有相同几何形状，输入倒位序、输出自然顺序的FFT流图

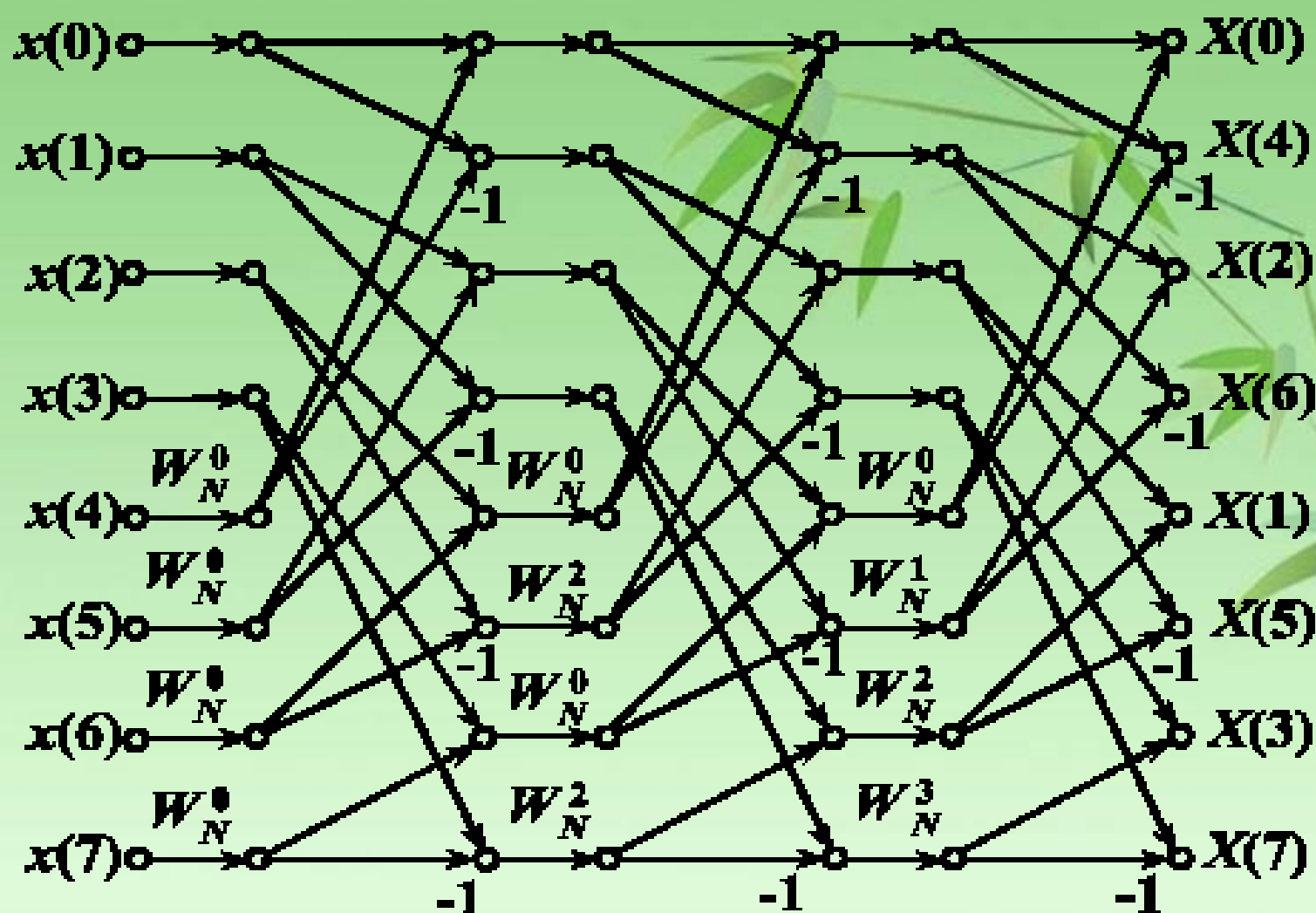


图 4-13 按时间抽选，各级具有相同几何形状，输入自然顺序、输出倒位序的FFT流图

例 用FFT算法处理一幅 $N \times N$ 点的二维图像，如用每秒可做10万次复数乘法的计算机，当 $N=1024$ 时，问需要多少时间（不考虑加法运算时间）？

解 当 $N=1024$ 点时，FFT算法处理一幅二维图像所需复数乘法约为 $\frac{N^2}{2} \log_2 N^2 \approx 10^7$ 次，仅为直接计算DFT所需时间的10万分之一。即原需要3000小时，现在只需要2 分钟。

4.3 按频率抽选的基-2FFT算法

（桑德—图基算法）

一、算法原理

- 对于 $N = 2^L$ 情况下另一种普遍使用的FFT结构是频率抽取法。
- 与DIT-FFT算法类似分解，但是抽取的是 $X(k)$ 。即分解 $X(k)$ 成奇数与偶数序号的两个序列。
- 设： $N = 2^L$ ， L 为整数。将 $X(k)$ 按 k 的奇偶分组前，先将输入 $x(n)$ 按 n 的顺序分成前后两半：

$$DFT : X(k) = DFT[x(n)] = \sum_{n=0}^{N-1} x(n) W_N^{kn}, 0 \leq k \leq N-1$$

$$\begin{aligned} X(k) &= \sum_{n=0}^{N-1} x(n) W_N^{nk} = \sum_{n=0}^{\frac{N}{2}-1} x(n) W_N^{nk} + \sum_{n=N/2}^{N-1} x(n) W_N^{nk} \\ &= \sum_{n=0}^{\frac{N}{2}-1} x(n) W_N^{nk} + \sum_{n=0}^{\frac{N}{2}-1} x(n + \frac{N}{2}) W_N^{(n+\frac{N}{2})k} \\ &= \sum_{n=0}^{\frac{N}{2}-1} \left[x(n) + (-1)^k x(n + \frac{N}{2}) \right] W_N^{nk} \quad 0 \leq k \leq N-1 \end{aligned}$$

$$\because W_N^{\frac{N}{2}k} = (-1)^k$$

$$X(k) = \sum_{n=0}^{\frac{N}{2}-1} \left[x(n) + (-1)^k x\left(n + \frac{N}{2}\right) \right] W_N^{nk} \quad 0 \leq k \leq N-1$$

按 k 的奇偶将 $X(k)$ 分成两部分： $\because W_N^{kN/2} = (-1)^k = \begin{cases} 1 & k \text{ 为偶数} \\ -1 & k \text{ 为奇数} \end{cases}$

下面讨论 $k = 2r, 2r+1$ 的 $X(k)$:

$$X(2r) = \sum_{n=0}^{\frac{N}{2}-1} \left[x(n) + x\left(n + \frac{N}{2}\right) \right] W_N^{2nr}$$

$$= \sum_{n=0}^{\frac{N}{2}-1} \left[x(n) + x\left(n + \frac{N}{2}\right) \right] W_{\frac{N}{2}}^{nr}$$

$$r = 0, 1, \dots, \frac{N}{2} - 1$$

$$X(2r+1) = \sum_{n=0}^{\frac{N}{2}-1} \left[x(n) - x\left(n + \frac{N}{2}\right) \right] W_N^{n(2r+1)}$$

$$= \sum_{n=0}^{\frac{N}{2}-1} \left\{ \left[x(n) - x\left(n + \frac{N}{2}\right) \right] W_N^n \right\} W_{\frac{N}{2}}^{nr}$$

这一结论表明：求
 $x(n)$ 的 N 点 DFT
 $X(k)$ 再次分解成 求
两个 $N/2$ 点 DFT

显然： $X(2r), X(2r+1)$ 对应两个 $N/2$ 点的 DFT。

$$\text{令: } \begin{cases} x_1(n) = x(n) + x(n + \frac{N}{2}) \\ x_2(n) = [x(n) - x(n + \frac{N}{2})] W_N^n \end{cases}$$

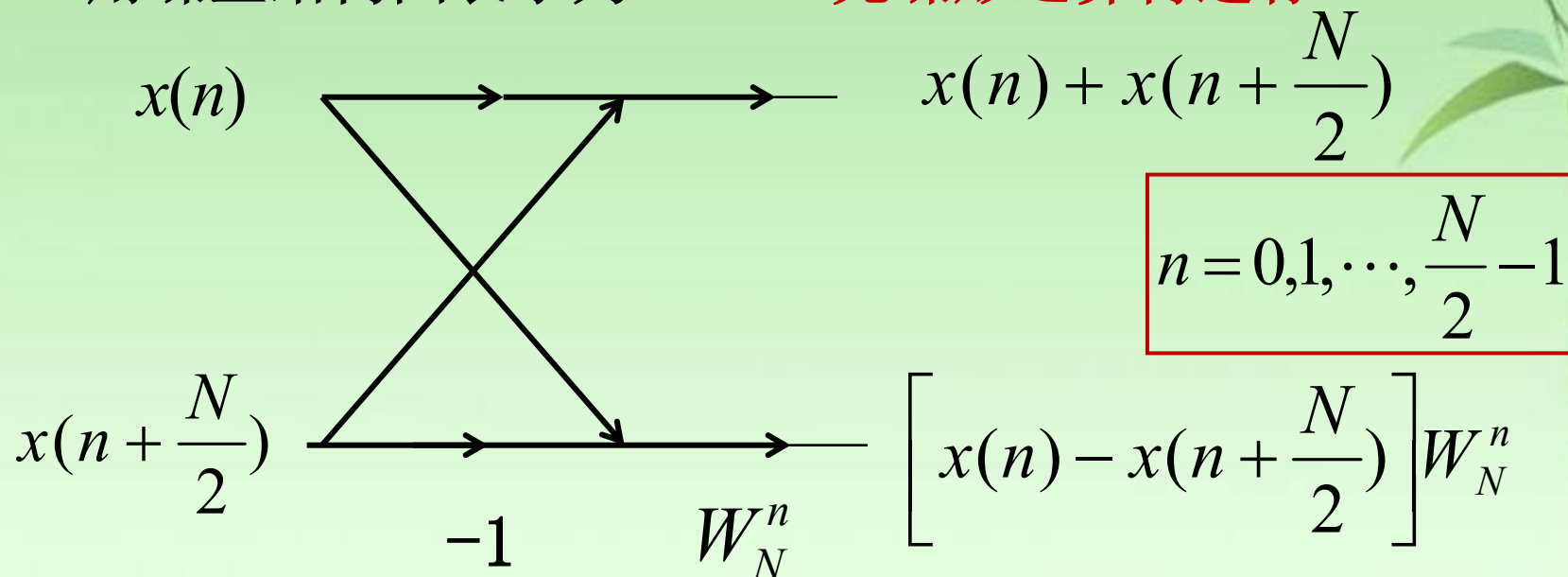
$$X(2r) == \sum_{n=0}^{\frac{N}{2}-1} x_1(n) W_N^{nr}$$

$$X(2r+1) == \sum_{n=0}^{\frac{N}{2}-1} x_2(n) W_N^{nr}$$

$$r = 0, 1, \dots, \frac{N}{2} - 1$$

用蝶型结构图表示为:

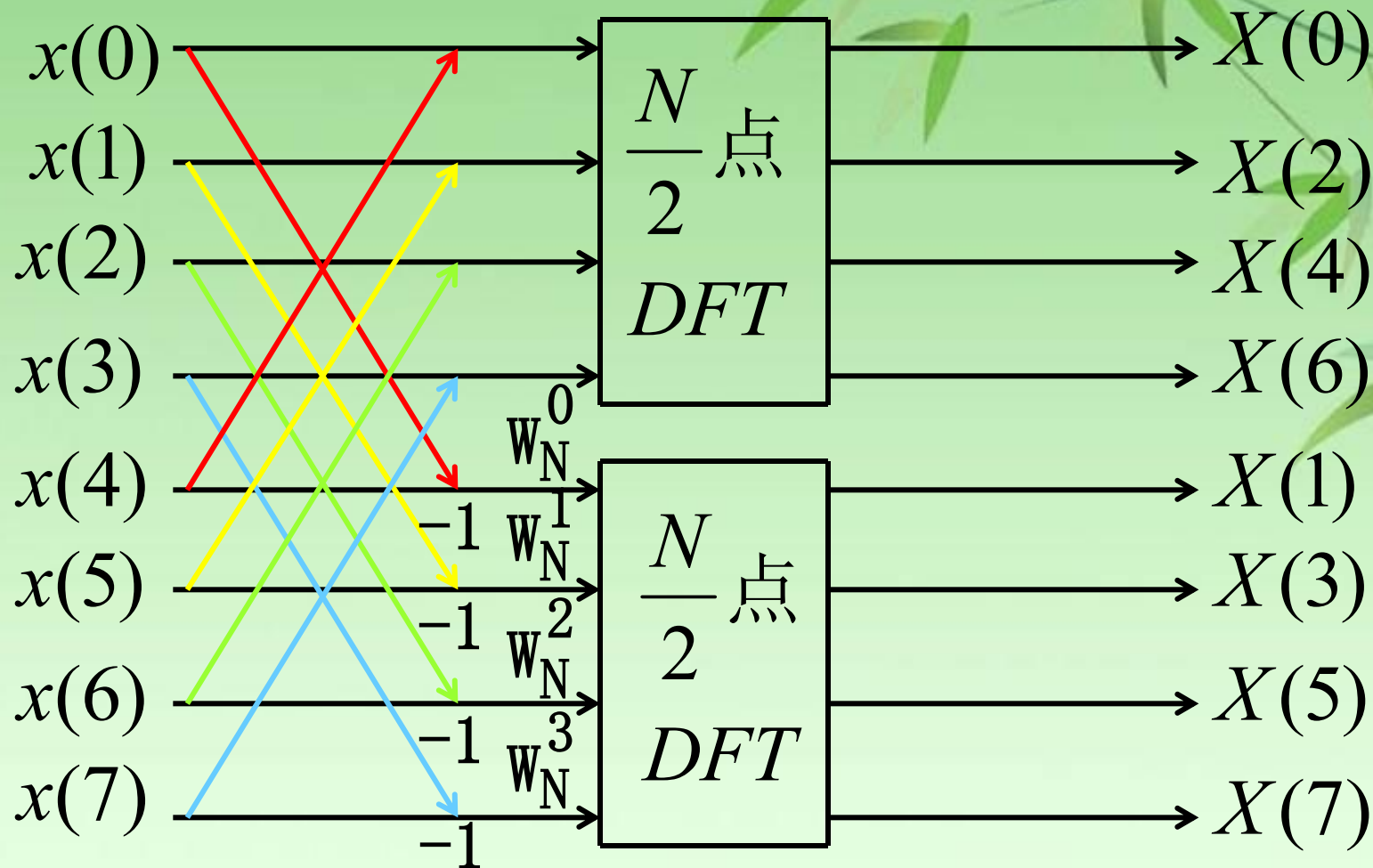
先蝶形运算再进行DFT



按频率抽取法的蝶形运算

按频率抽取的FFT-8点FFT例子

DIF-FFT的一次分解运算流图(N=8)

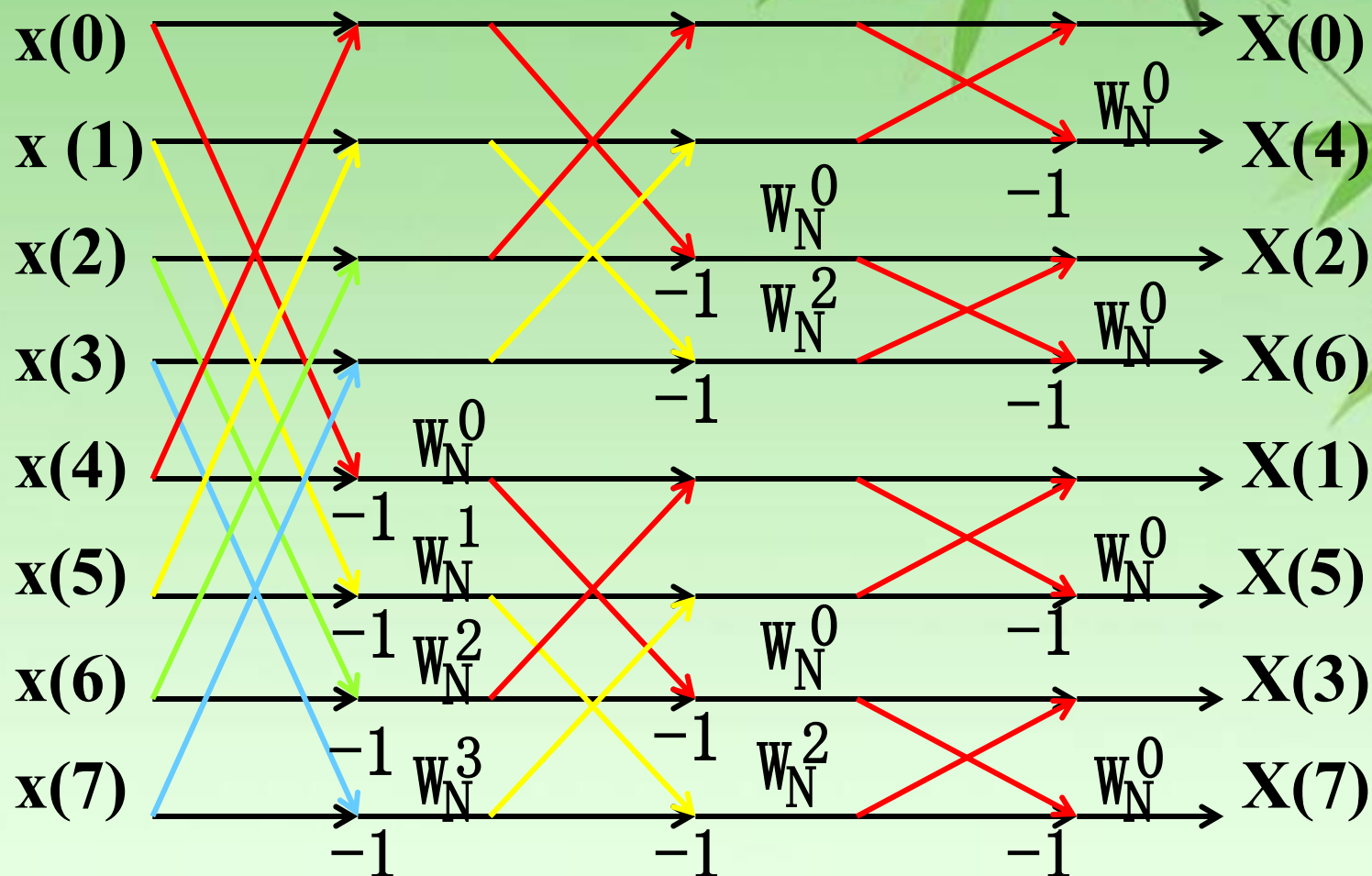


两个N/2点的DFT(先蝶式运算, 后 DFT)

N/2仍为偶数，进一步分解： $N/2 \rightarrow N/4$ 按照以上思路继续将 $N/2$ 点DFT分成偶数组和奇数组，即一个 $N/2$ 的DFT分解成两个 $N/4$ 点DFT，其输入序列分别是 $x_1(n)$ 和 $x_2(n)$ 按**上下对半分开后**通过蝶式运算构成的 **4** 个子序列，经过 **L** 次分解，最后分解为 **$N/2$ 个两点DFT**，它只有加减运算，但是为了比较并统一运算结构，我们仍然采用系数为 W_N^0 ，这 $N/2$ 个2点DFT的输出就是 N 点DFT的结果 $X(k)$ ，这就是**DIF-FFT算法**。

按频率抽取的FFT-8点FFT例子

DIF-FFT的L次分解运算流图(N=8)



二、按频率抽取FFT运算特点

•运算量

虽然蝶形结构不同于按时间抽取FFT,但单个蝶形运算量相同,依然是2次加法,一次乘法,总运算量与按时间抽取算法相同。

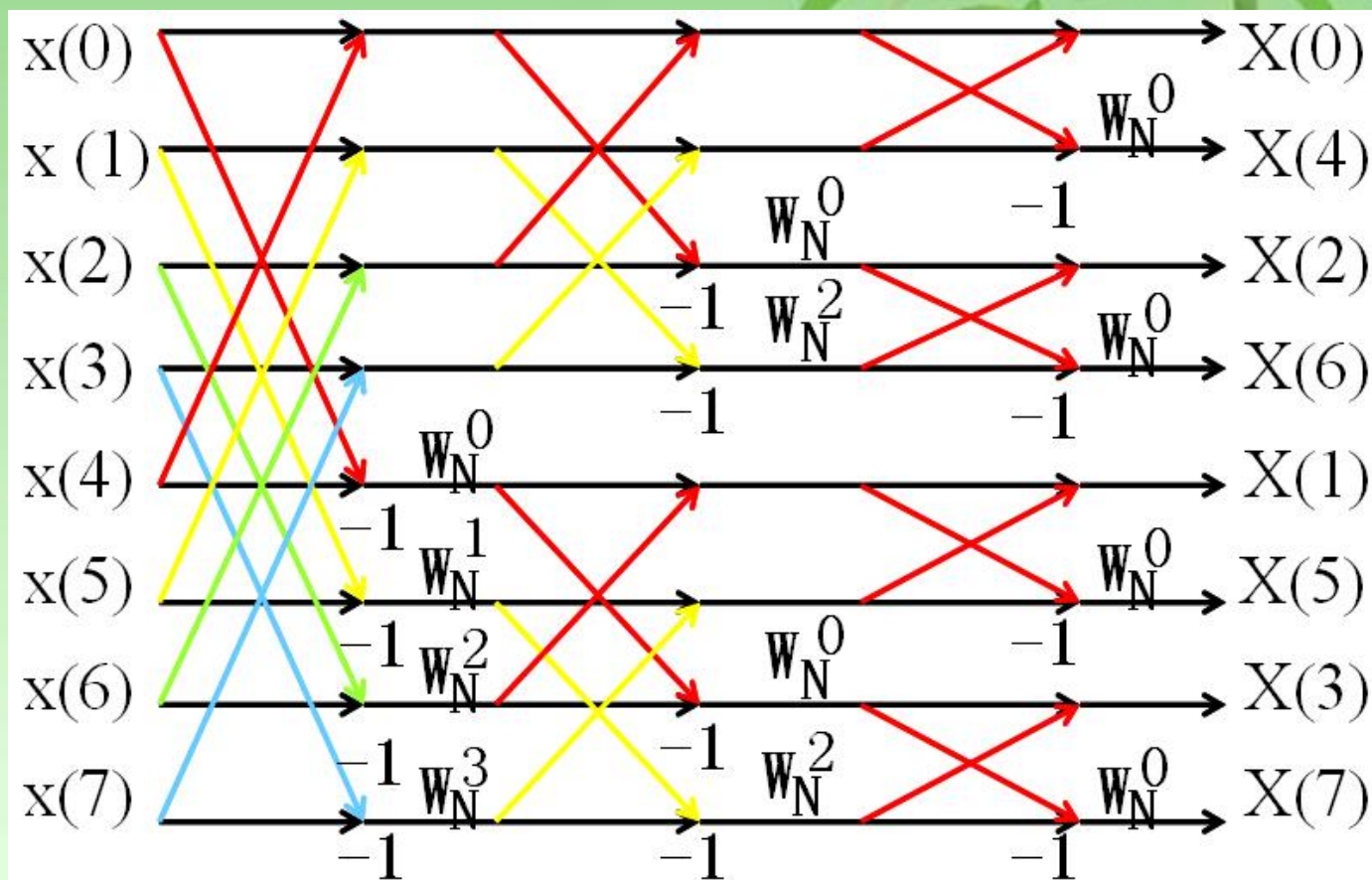
$N=2^L$ 时, 共有 $L=\log_2 N$ 级运算; 每一级有 $N/2$ 个蝶形运算, 每个蝶形运算有一次复乘,两次复加。共有 $N/2\log_2 N$ 个蝶形运算。

因此, N 点的FFT的运算量为

$$\text{复乘: } m_F = \frac{N}{2} L = \frac{N}{2} \log_2 N$$

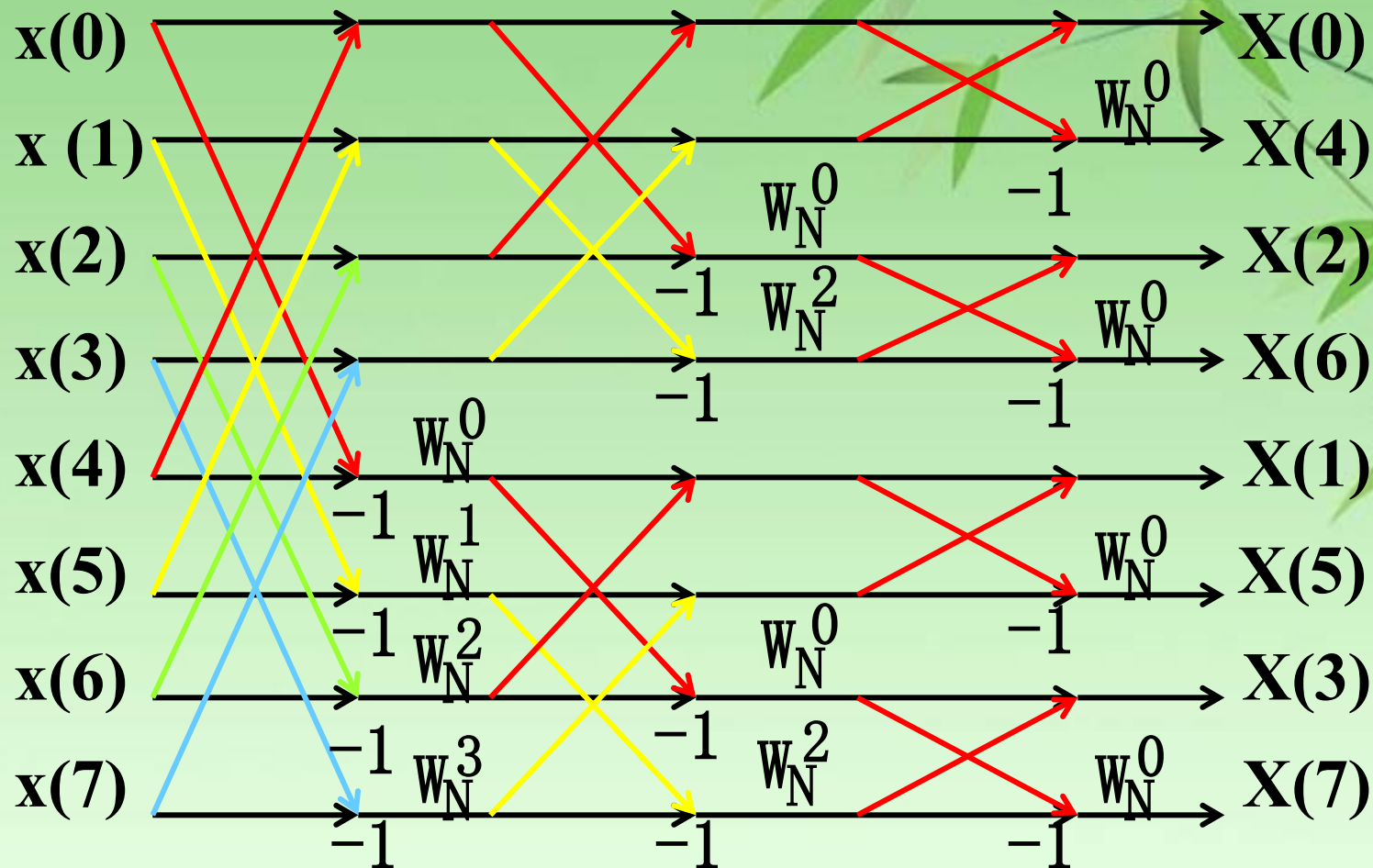
$$\text{复加: } a_F = NL = N \log_2 N$$

•序数重排 输入自然序，输出倒位序



输入正好是自然顺序,而输出是以按位反转的规律重排的

•原位计算

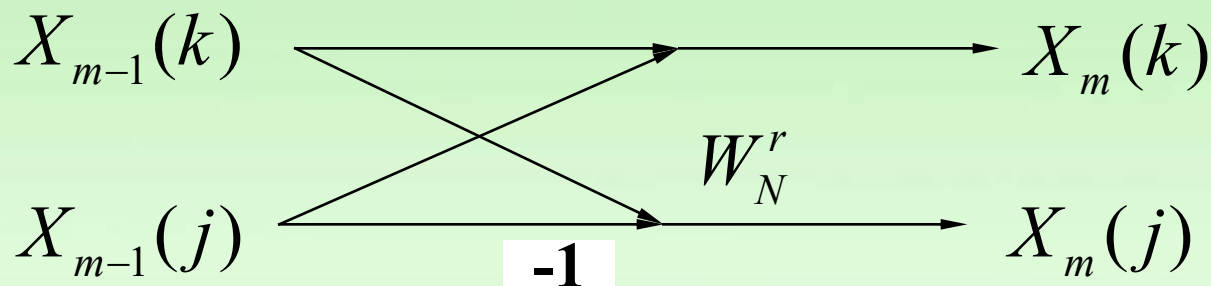


频率抽取法也是可以实现**原位运算**,即输入数据,中间结果,最终结果可以存储于同一个单元。这是时间抽取FFT和频率抽取FFT的**相同点**。

L 级蝶形运算，每级 **$N/2$** 个蝶形，每个蝶形结构：

$$\begin{cases} X_m(k) = X_{m-1}(k) + X_{m-1}(j) \\ X_m(j) = [X_{m-1}(k) - X_{m-1}(j)]W_N^r \end{cases}$$

m 表示第 m 级迭代， k, j 表示数据所在的行数



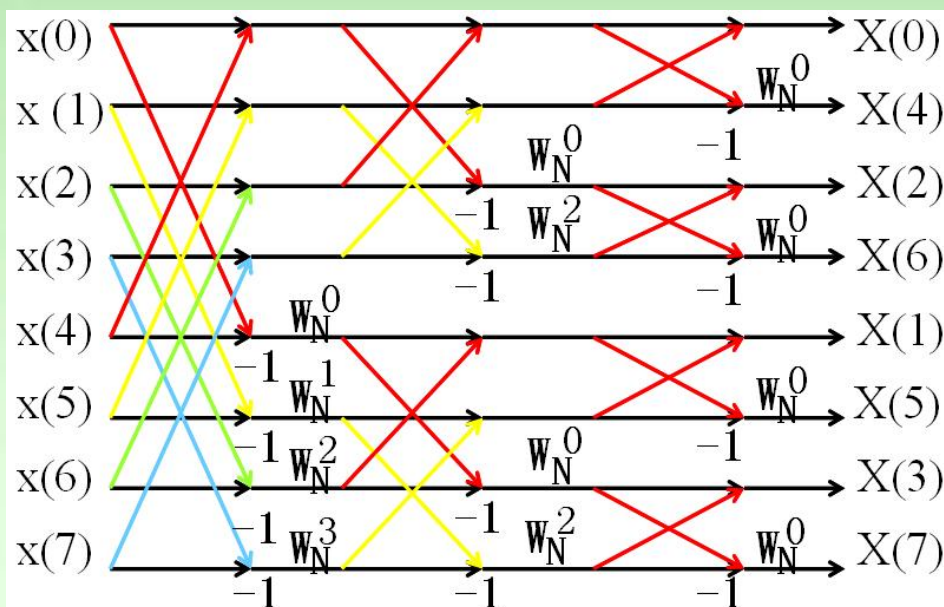
•蝶形运算

对 $N=2^L$ 点FFT，输入自然序，输出倒位序，

两节点距离： $2^{L-m}=N / 2^m$

第 m 级运算：

$$\begin{cases} X_m(k) = X_{m-1}(k) + X_{m-1}(k + \frac{N}{2^m}) \\ X_m(k + \frac{N}{2^m}) = [X_{m-1}(k) - X_{m-1}(k + \frac{N}{2^m})]W_N^r \end{cases}$$

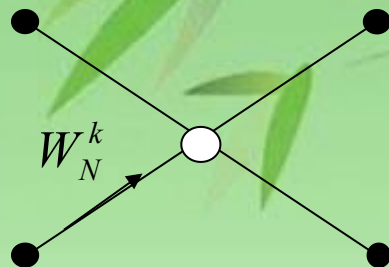


•蝶形类型随迭代次数成倍减少，每迭代一次，蝶形类型减少一倍，数据点间隔也减少一倍。

•第 m 级运算有 2^{L-m} 个蝶形运算系数/蝶形类型和取数间隔

W_N^r 的确定:

第 m 级的 r 取值:



- 蝶形运算两节点的第一个节点为 k 值，表示成 L 位二进制数，左移 $m-1$ 位，把右边空出的位置补零，结果为 r 的二进制数。

$$r = (k)_2 \cdot 2^{m-1}$$

$$r = k \cdot 2^{m-1}$$

$$k = 0, 1, 2, \dots, 2^{L-m} - 1$$

存储单元

输入序列 $x(n)$: N 个存储单元

系数 W_N^r : $N/2$ 个存储单元

DIT法与DIF法的异同

- 原位运算
- 运算量相同
- 序数重排
- 蝶形运算

DIT法与DIF法是两种等价的FFT运算

❑ 运算量相同

$$\text{复乘: } m_F = \frac{N}{2}L = \frac{N}{2}\log_2 N$$

❑ 存储量相同

$$\text{复加: } a_F = NL = N\log_2 N$$

❑ 原位运算

❑ 序数重排

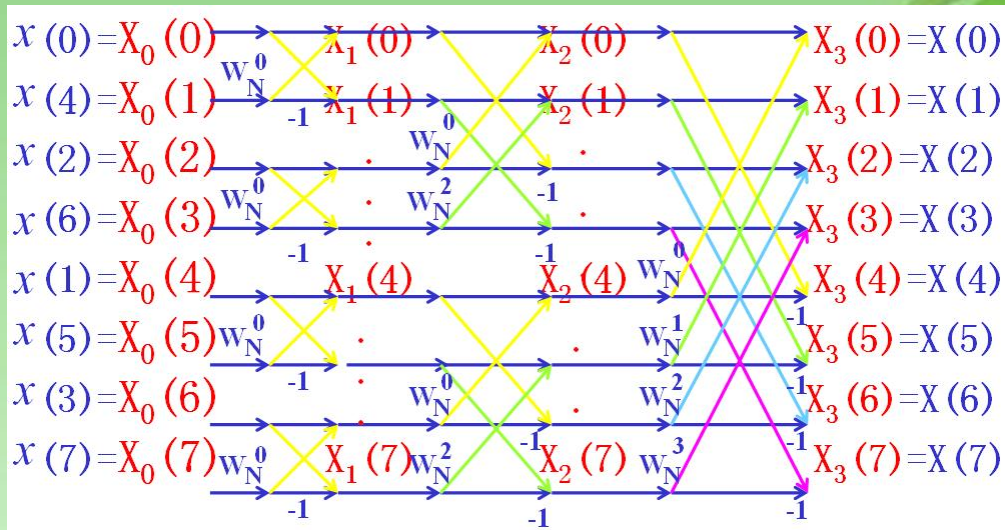
❑ 蝶式运算过程不同

❑ *DIT*: 先复乘后加减

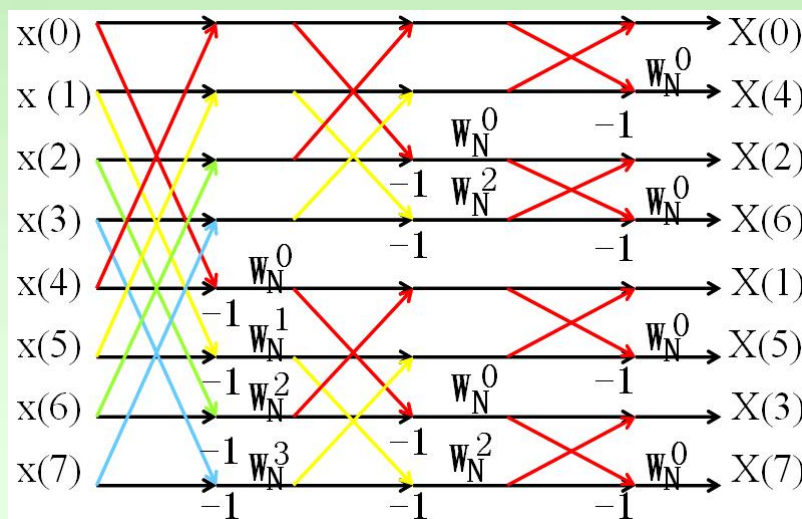
❑ *DIF*: 先减后复乘

❑ *DIT*和*DIF*的基本蝶形互为转置

•DIT与DIF算法的流图对比



- 输入变输出
- 输出变输入
- 流图反转



DIT法与DIF法
是两种等价的FFT运算

4.5 离散傅里叶反变换IDFT的快速算法IFFT (FFT应用一)

一、从定义比较分析

$$x(n) = IDFT[X(k)] = \frac{1}{N} \sum_{k=0}^{N-1} X(k) W_N^{-kn}$$

$$X(k) = DFT[x(n)] = \sum_{n=0}^{N-1} x(n) W_N^{kn}$$

与DFT的比较:

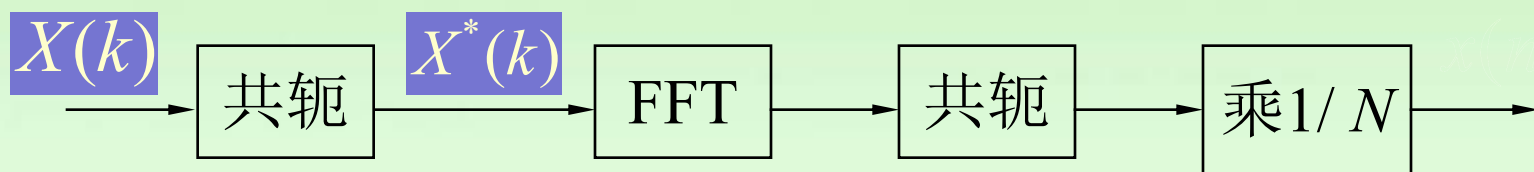
- 1) 旋转因子 W_N^{-kn} 的不同;
- 2) 结果还要乘 $1/N$ 。

二、实现算法——直接使用FFT程序的算法

把 $X^*(k)$ 作为输入，用一个FFT可得到 $N x^*(n)$ ，对此再求复共扼并除以 N 就得到所需的反变换 $x(n)$ 。

$$x^*(n) = \{IDFT[X(k)]\}^* = \frac{1}{N} \sum_{k=0}^{N-1} X^*(k) W_N^{kn} = \frac{1}{N} \underbrace{\sum_{k=0}^{N-1} X^*(k) W_N^{kn}}_{DFT[X^*(k)]}$$
$$\Rightarrow x(n) = \frac{1}{N} \{DFT[X^*(k)]\}^*$$

直接调用FFT子程序计算IFFT的方法：



这种方法虽然用了两次取共扼运算，但可以用**同一个FFT子程序**，因而用起来很方便。

4.9 线性卷积的FFT算法 (FFT应用三)

一、基本算法思路

若 L 点 $x(n)$, M 点 $h(n)$,

则直接计算其线性卷积 $y(n)$

$$y(n) = x(n) * h(n) = \sum_{m=0}^{M-1} h(m)x(n-m)$$

需运算量: $M_d = LM$ (乘法次数)

若系统满足线性相位, 即:

$$h(n) = \pm h(M-1-n)$$

则需运算量: $m_d = LM/2$

FFT法：以圆周卷积代替线性卷积

$$\text{令 } N = 2^m \geq M + L - 1$$

$$x(n) = \begin{cases} x(n) & 0 \leq n \leq L-1 \\ 0 & L \leq n \leq N-1 \end{cases}$$

$$h(n) = \begin{cases} h(n) & 0 \leq n \leq M-1 \\ 0 & M \leq n \leq N-1 \end{cases}$$

$$\text{则 } y(n) = x(n) * h(n) = x(n) \textcircled{N} h(n)$$

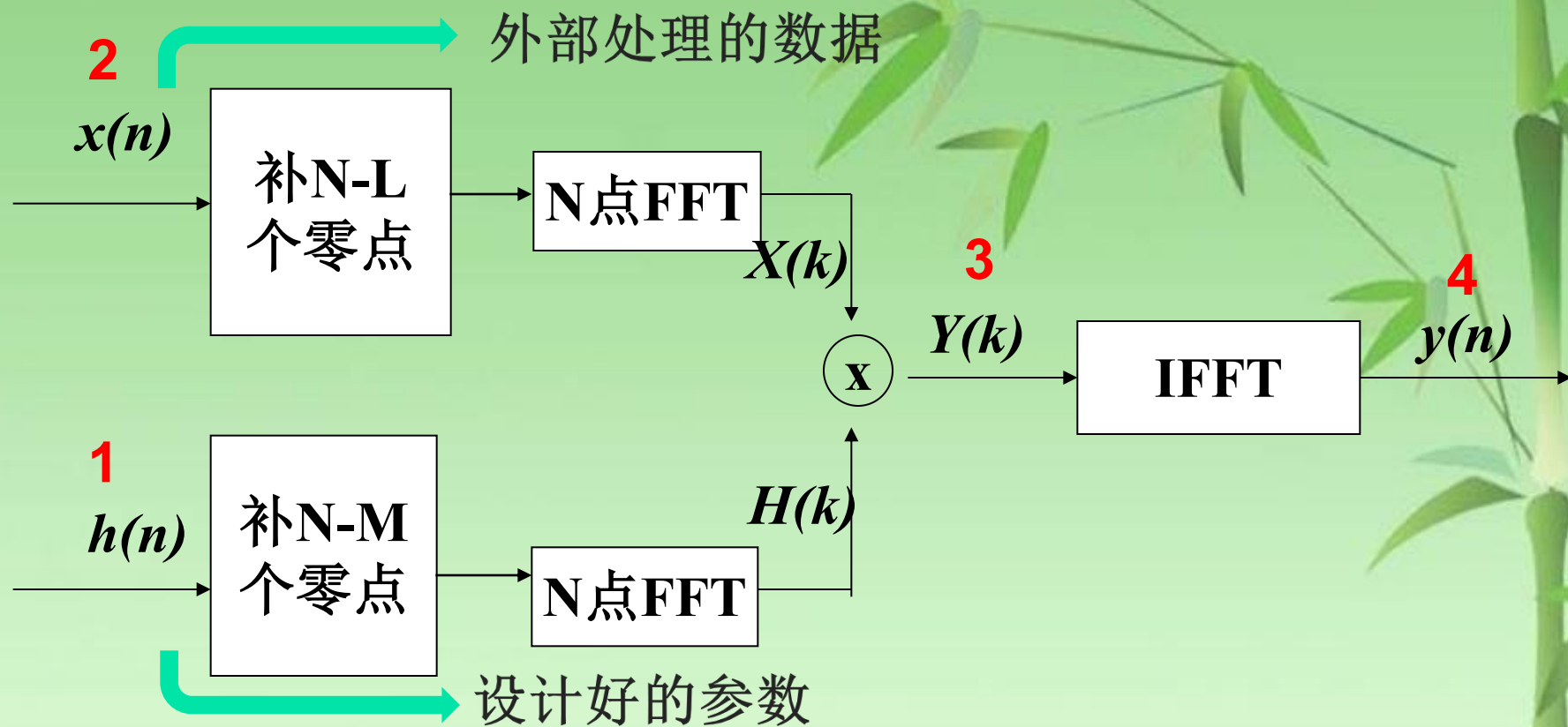
$$1) H(k) = FFT[h(n)] \quad N/2 * \log_2 N$$

$$2) X(k) = FFT[x(n)] \quad N/2 * \log_2 N$$

$$3) Y(k) = H(k)X(k) \quad N$$

$$4) y(n) = IFFT[Y(k)] \quad N/2 * \log_2 N$$

$$m_F = N(1 + 3/2 * \log_2 N)$$



直接计算卷积乘法次数： $L \cdot M$

循环卷积的乘法次数： $\frac{3}{2}N \cdot \log_2 N + N$ 三个FFT和N个乘法

比较直接计算和FFT法计算的运算量

$$K_m = \frac{m_d}{m_F} = \frac{ML}{2N(1 + 3/2 * \log_2 N)}$$

讨论：① **x(n)**和**h(n)**长度差不多的情况

当 $M \approx L$ 则 $N = M + L - 1 \approx 2M$

$$K_m = \frac{M^2}{4M[1 + 3/2 * (1 + \log_2 M)]} = \frac{M}{10 + 6\log_2 M}$$

L=M	8	32	64	128	256	512	1024	2048	4096
K _m	0.286	0.080	1.39	2.46	4.41	8	14.62	26.95	49.95

可见，当**L**越大，循环卷积优越性越大，故用循环卷积计算线性卷积通常也称为**快速卷积**

② 当 $x(n)$ 很长

当 $L \gg M$ 则 $N = M + L - 1 \approx L$

$$K_m = \frac{M}{2 + 3 \log_2 L}$$

$L \uparrow \uparrow$ $K_m \downarrow$ 需采用分段卷积

$\left\{ \begin{array}{l} \text{重叠相加法} \\ \text{重叠保留法} \end{array} \right.$

L太大，循环卷积优越性不能充分发挥出来。**(h(n)补很多零，很不经济)**

L太大， $x(n)$ 占用大量的存储单元和处理时间的延误，对系统性能影响不利。

克服困难的办法——采用分段卷积的办法。将 $x(n)$ 分成和 $h(n)$ 相仿的片段。

1、重叠相加法(分段卷积的各段相加构成总的卷积输出)

$h(n)$ 的长度为 M 点序列 ($0 \leq n \leq M-1$)， $x(n)$ 长度很长时，将 $x(n)$ 分为 L 长的若干小的片段， L 与 M 可比拟(等数量级)。

$$x_i(n) = \begin{cases} x(n), & iL \leq n \leq (i+1)L - 1 \\ 0, & \text{其它 } n \end{cases}$$

则：

$$x(n) = \sum_{i=-\infty}^{\infty} x_i(n)$$

输出：

$$\begin{aligned} y(n) &= x(n) * h(n) = \sum_{i=-\infty}^{\infty} x_i(n) * h(n) \\ &= \sum_{i=-\infty}^{\infty} y_i(n) \end{aligned}$$

其中： $y_i(n) = x_i(n) * h(n)$

可以用**圆周卷积**计算：

$$y_i(n) = x_i(n) \textcircled{\text{N}} h(n) \quad N = L + M - 1$$

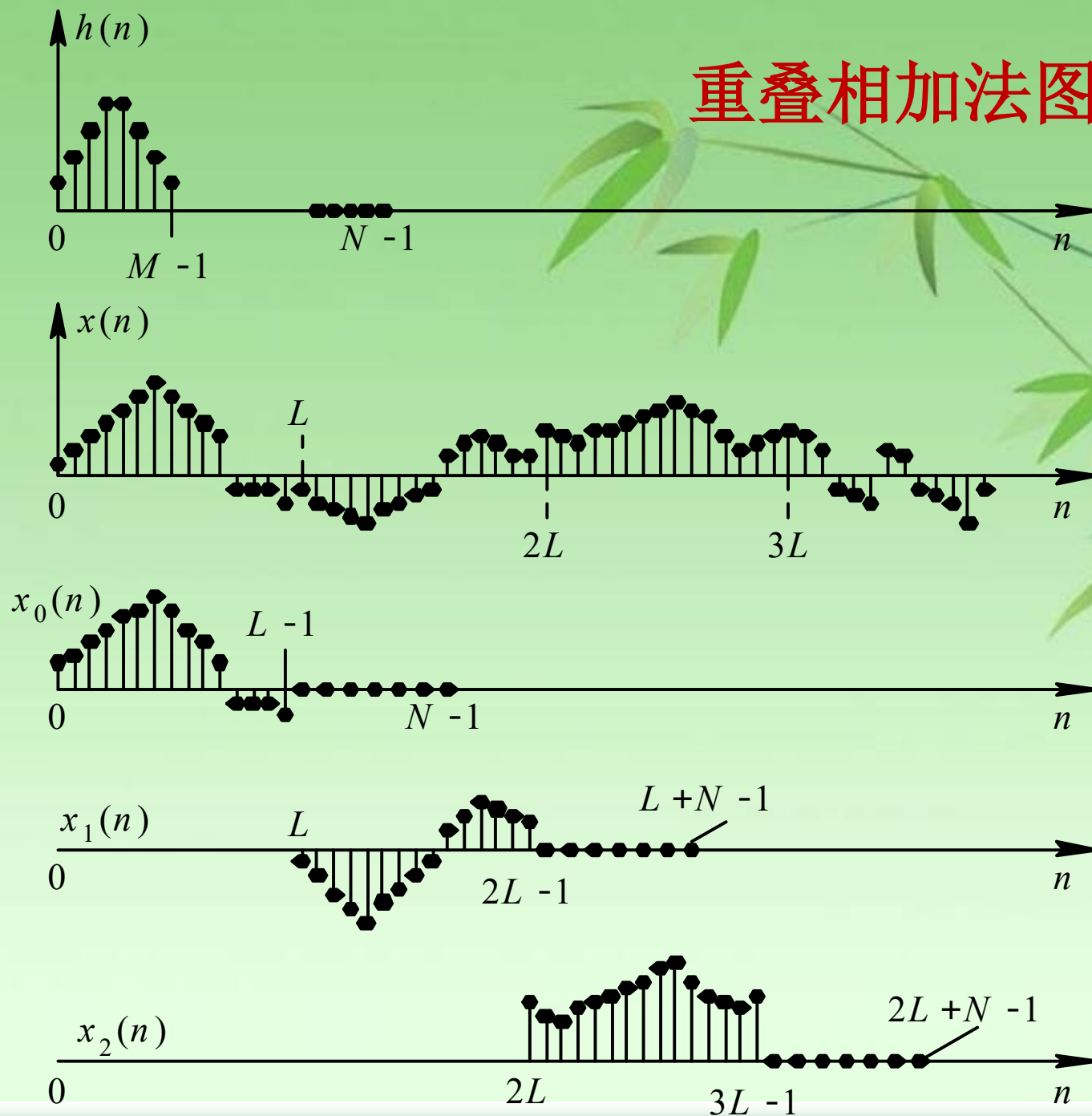
选 $N = 2^m$, 上面圆周卷积可用**FFT**计算。

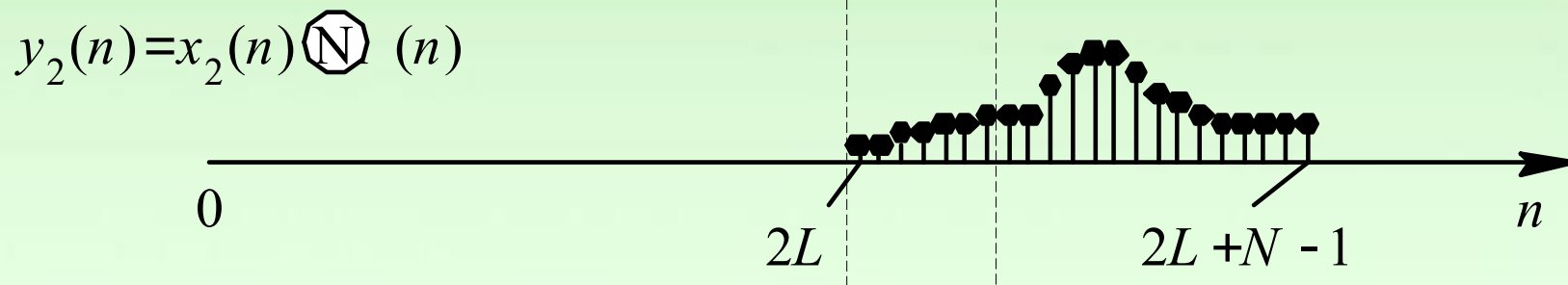
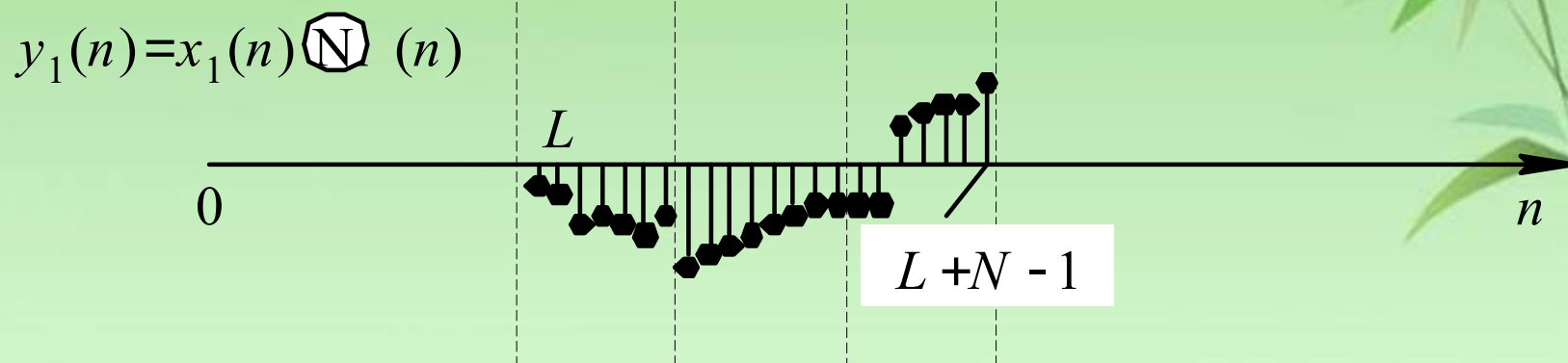
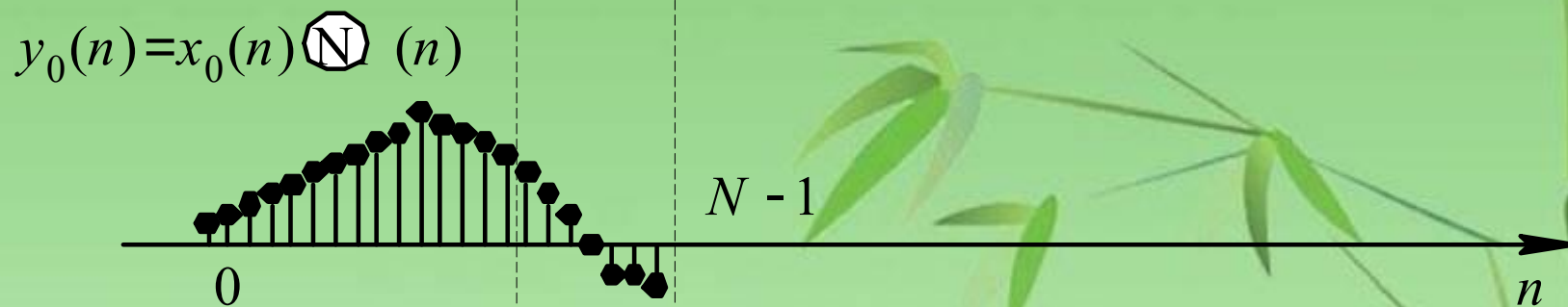
采用基二FFT算法，N应满足 **$(L+M-1)=N=2^m$** , **m**通常取**8、9、10**，**N**为**256、512、1024**

由于 $y_i(n)$ 长度为 N ，而 $x_i(n)$ 长度 L ，必有 **$M-1$ 点重叠**， $y_i(n)$ 应相加才能构成最后 $y(n)$ 的。

$$y(n) = \sum_{i=-\infty}^{\infty} y_i(n)$$

重叠相加法图形





重叠相加法图形

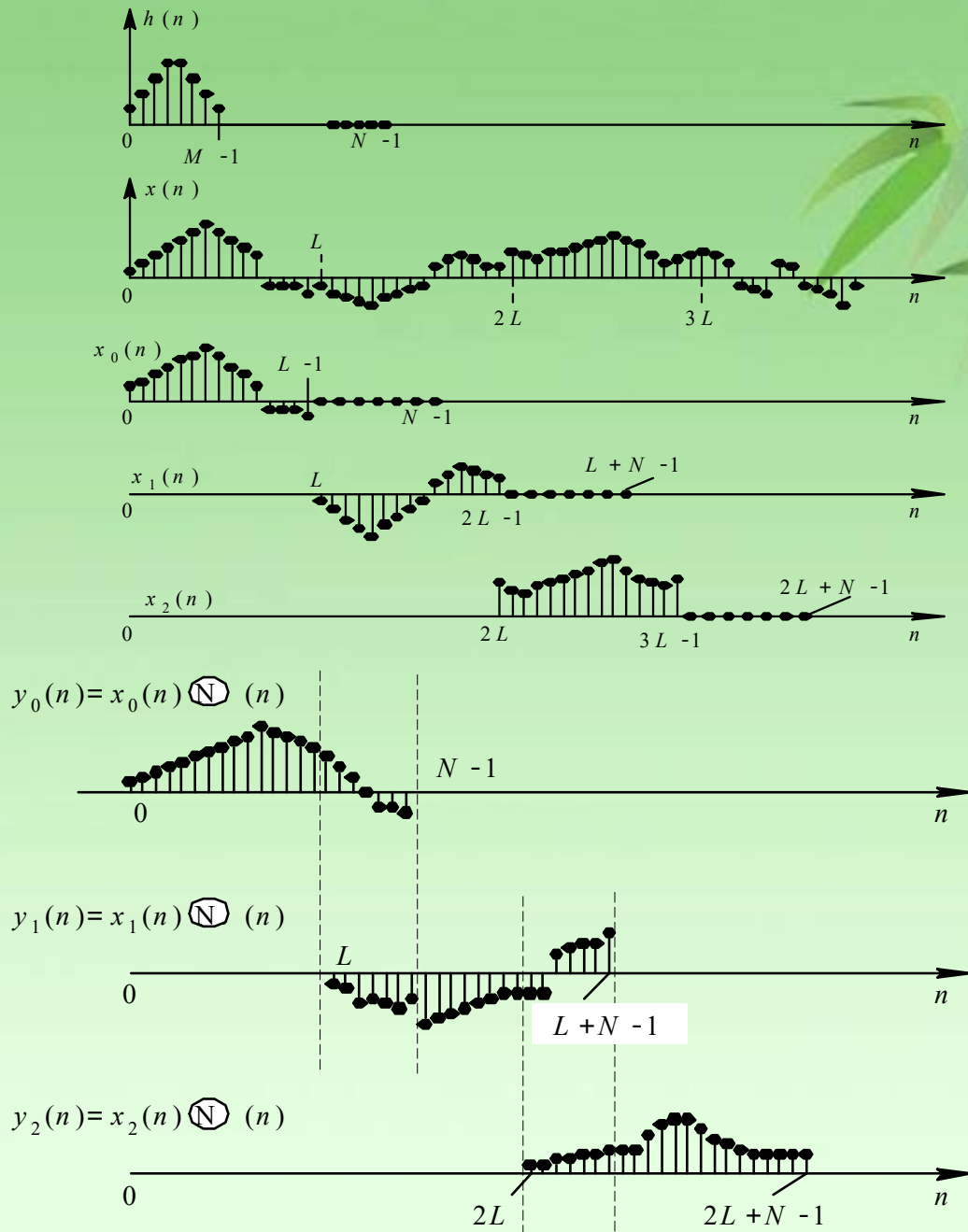
每项 $x_i(n)$ 只有L个非零样本

$h(n)$ 的长度为M

线性卷积 $y_i(n)$ 的长度为(L+M-1)，相邻两端 $y_i(n)$ 序列必然有(M-1)点的部分发生重叠，这个重叠部分加起来才能构成最后的输出序列 $y_i(n)$

❖ 显然可用快速卷积法计算分段卷积，快速卷积的计算区间为 $N = L + M - 1$ 。

❖ 这种方法不要求大的存贮容量，而且运算量和延时也大大减少。



和上面的讨论一样，用**FFT**法实现重叠相加法的步骤如下：

① 计算 N 点FFT， $H(k)=\text{DFT} [h(n)]$ ；

② 计算 N 点FFT， $X_i(k)=\text{DFT} [x_i(n)]$ ；

③ 相乘， $Y_i(k)=X_i(k)H(k)$ ；

④ 计算 N 点IFFT， $y_i(n)=\text{IDFT} [Y_i(k)]$ ；

⑤ 将各段 $y_i(n)$ （包括**重叠部分**）**相加**， $y(n) = \sum_{i=0}^{\infty} y_i(n)$ 。

重叠相加的名称是由于**各输出段的重叠部分相加**而得名的。

例 已知序列 $x[n]=n+2, 0 \leq n \leq 12$, $h[n]=\{1,2,1\}$ 试利用重叠相加法计算线性卷积, 取 $L=5$ 。

解: 重叠相加法

$$x_1[n]=\{2, 3, 4, 5, 6\} \quad x_2[n]=\{7, 8, 9, 10, 11\} \quad x_3[n]=\{12, 13, 14\}$$

$$y_1[n]=\{2, 7, 12, 16, 20, 17, 6\}$$

$$y_2[n]=\{7, 22, 32, 36, 40, 32, 11\}$$

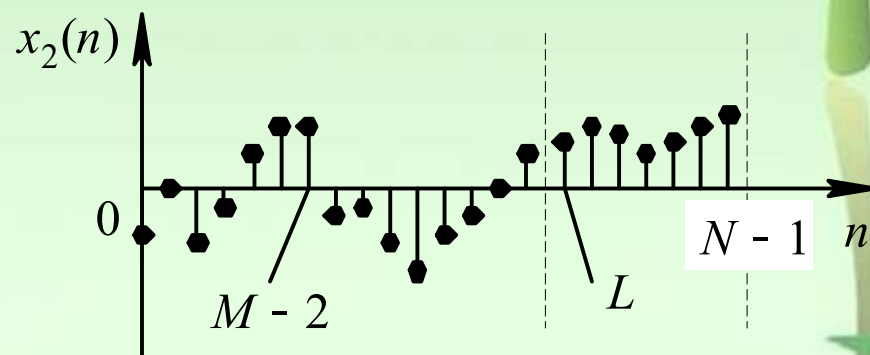
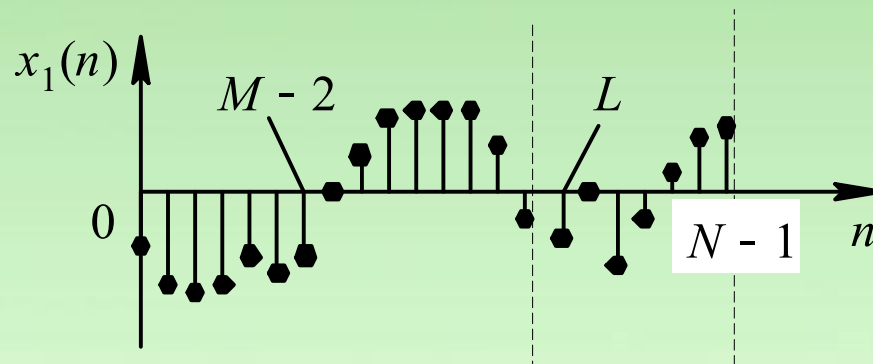
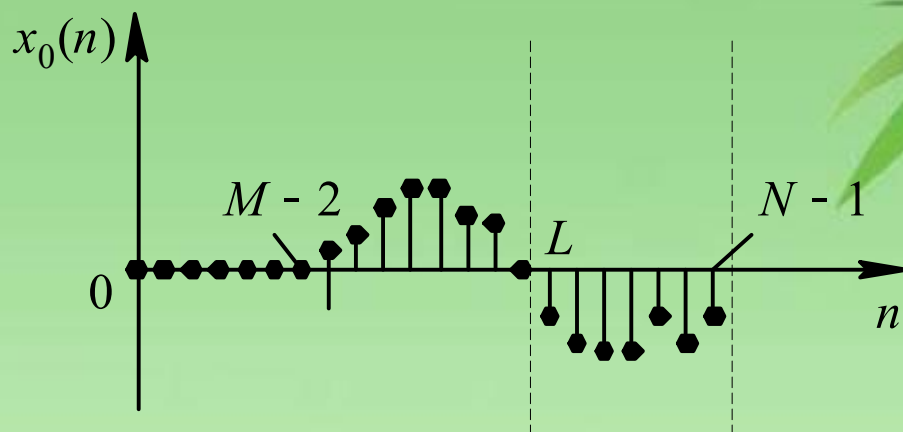
$$y_3[n]=\{12, 37, 52, 41, 14\}$$

$$y[n]=\{2, 7, 12, 16, 20, 24, 28, 32, 36, 40, 44, 48, 52, 41, 14\}$$

2、重叠保留法(重叠了输入信号，省略了输出端的重叠相加)

- 此方法与上述方法稍有不同。
- 先将 $x(n)$ 分段，每段 $L=N-M+1$ 个点，这是相同的。
- 不同之处是，序列中补零处不补零，而在每一段的前边补上前一段保留下来的 $(M-1)$ 个输入序列值，组成 $L+M-1$ 点序列 $x_i(n)$ 。
- 如果 $L+M-1 < 2^m$ ，则可在每段序列末端补零值点，补到长度为 2^m ，这时如果用DFT实现 $h(n)$ 和 $x_i(n)$ 圆周卷积，则其每段圆周卷积结果的前 $(M-1)$ 个点的值不等于线性卷积值，必须舍去。
- 重叠保留法与重叠相加法的计算量差不多，但省去了重叠相加法最后的相加运算。

重叠保留法示意图

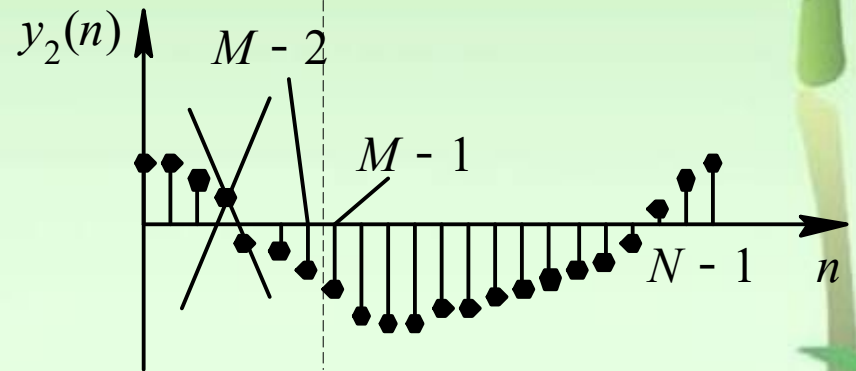
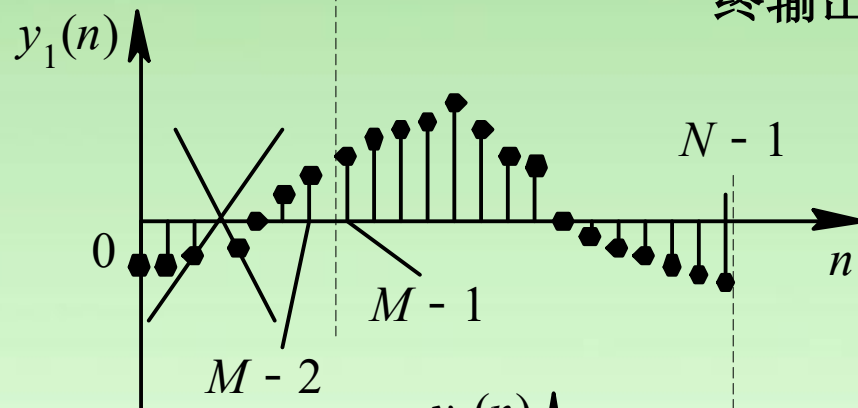
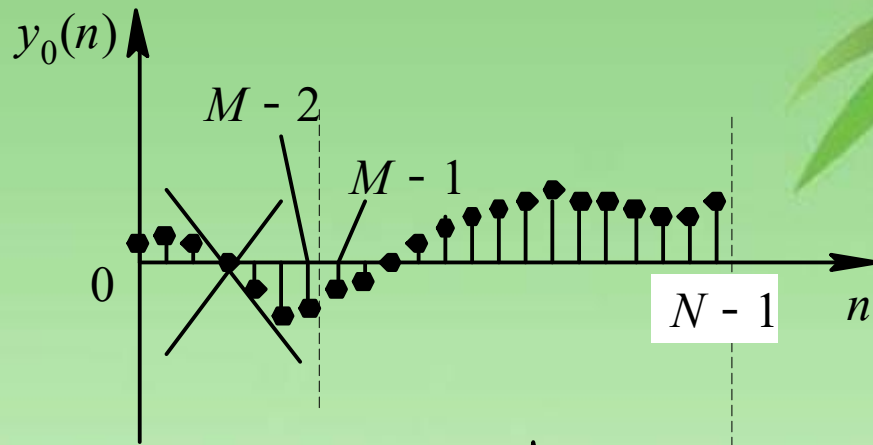


(a)

重叠保留法示意图

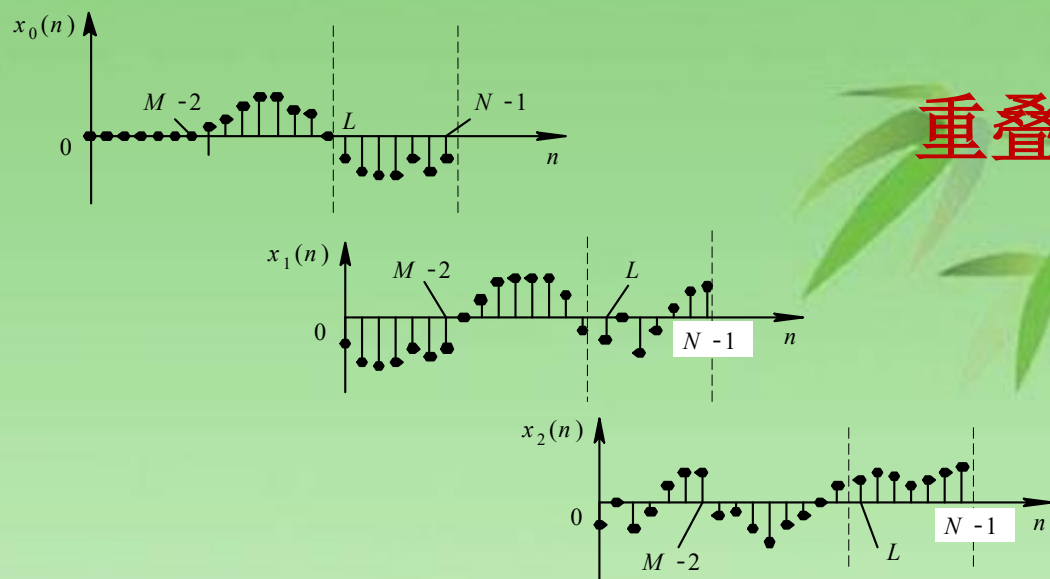
去掉每一输出段 $y_i(n)$ 的前 $(M-1)$ 个点，**衔接**相邻段留下来的点，构成最终输出

(b)

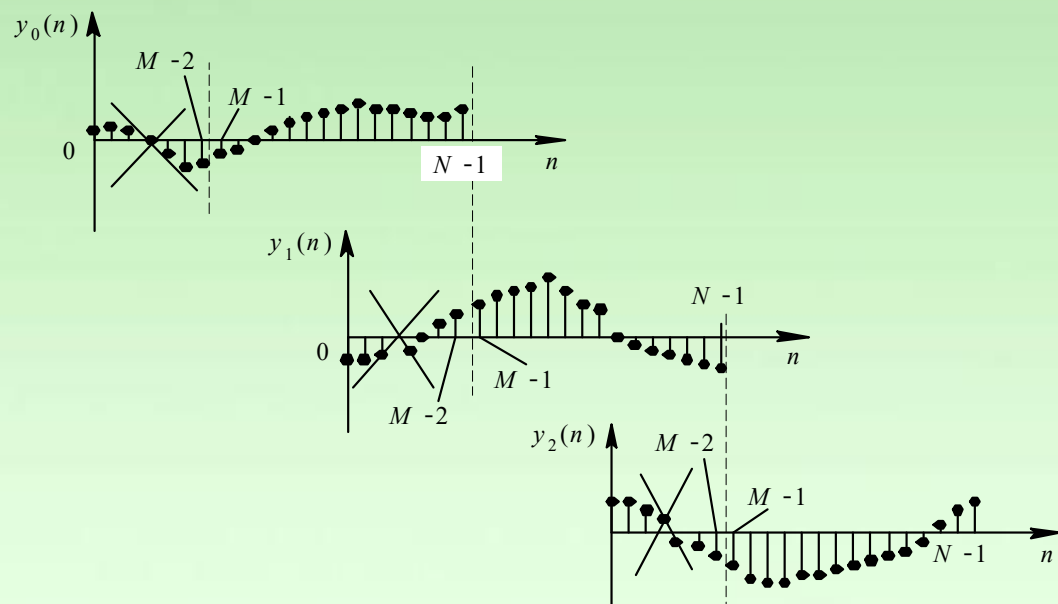


重叠保留法示意图

(a)



(b)



去掉每一输出段 $y_i(n)$ 的前 $(M-1)$ 个点，衔接相邻段留下来的点，构成最终输出

例 已知序列 $x[n]=n+2, 0 \leq n \leq 12$, $h[n]=\{1,2,1\}$ 试利用重叠保留法计算线性卷积, 取 $L=5$ 。

解: 重叠保留法

$$x_1[k]=\{0, 0, 2, 3, 4\} \quad x_2[k]=\{3, 4, 5, 6, 7\} \quad x_3[k]=\{6, 7, 8, 9, 10\}$$

$$x_4[k]=\{9, 10, 11, 12, 13\} \quad x_5[k]=\{12, 13, 14, 0, 0\}$$

$$y_1[k]=x_1[k] \otimes h[k]=\{\underline{11}, \underline{4}, 2, 7, 12\}$$

$$y_2[k]=x_2[k] \otimes h[k]=\{\underline{23}, \underline{17}, 16, 20, 24\}$$

$$y_3[k]=x_3[k] \otimes h[k]=\{\underline{35}, \underline{29}, 28, 32, 36\}$$

$$y_4[k]=x_4[k] \otimes h[k]=\{\underline{47}, \underline{41}, 40, 44, 48\}$$

$$y_5[k]=x_5[k] \otimes h[k]=\{\underline{12}, \underline{37}, 52, 41, 14\}$$

$$y[k]=\{2, 7, 12, 16, 20, 24, 28, 32, 36, 40, 44, 48, 52, 41, 14\}$$